

Rethinking System Software for Efficient, Elastic Cloud Computing

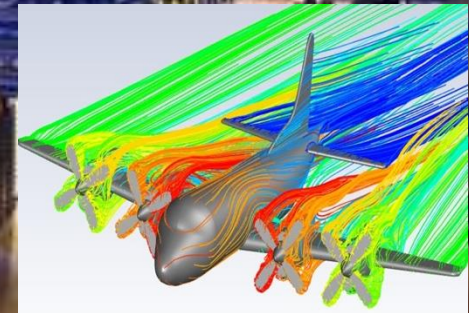
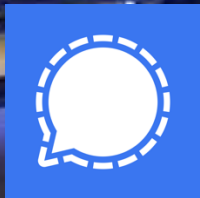
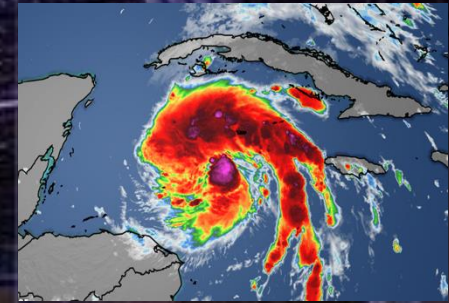
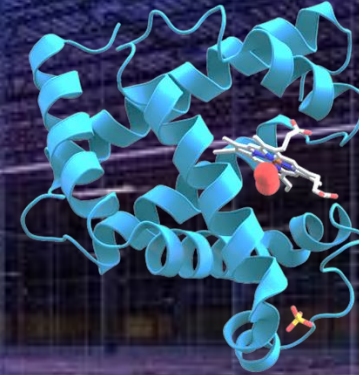
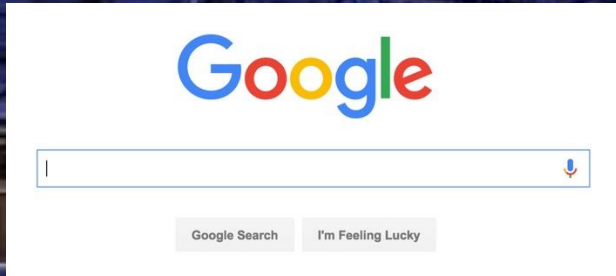
Ana Klimovic

ETH*zürich*

NUS Research Week
January 8, 2025



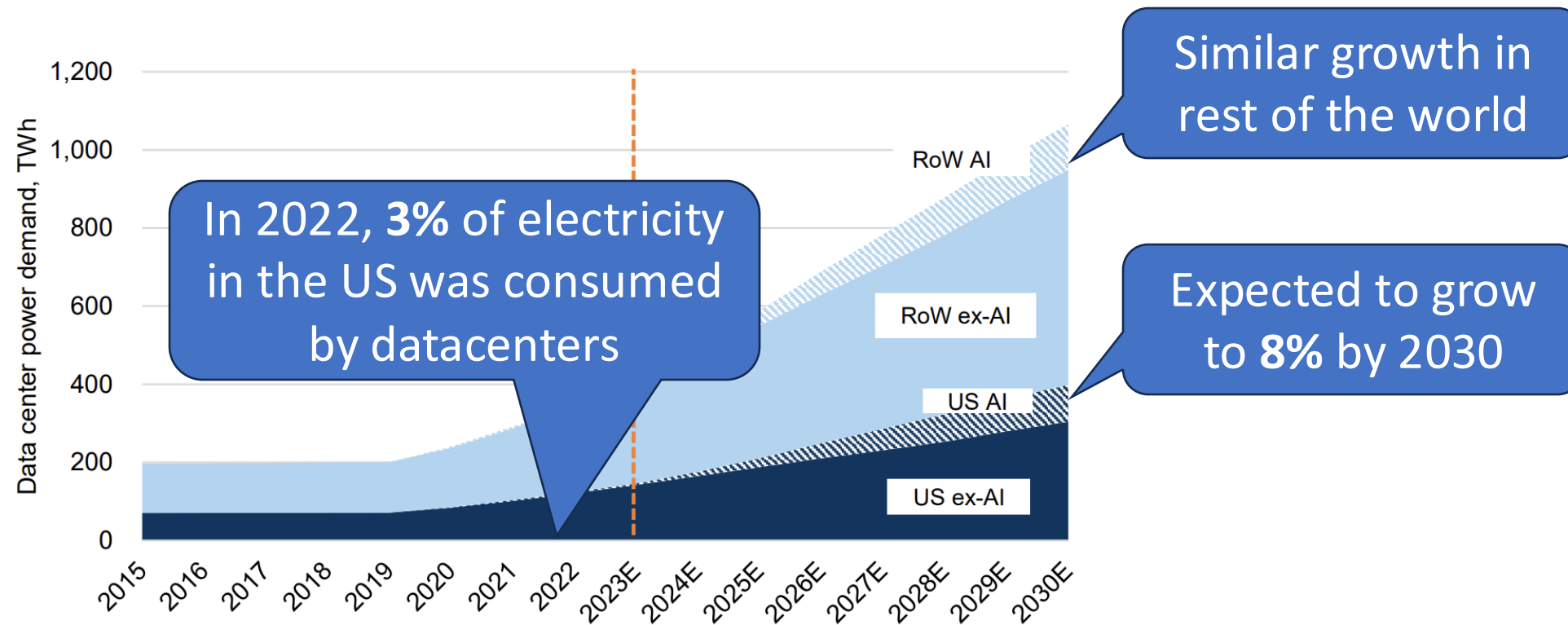




Cloud computing consumes a lot of power

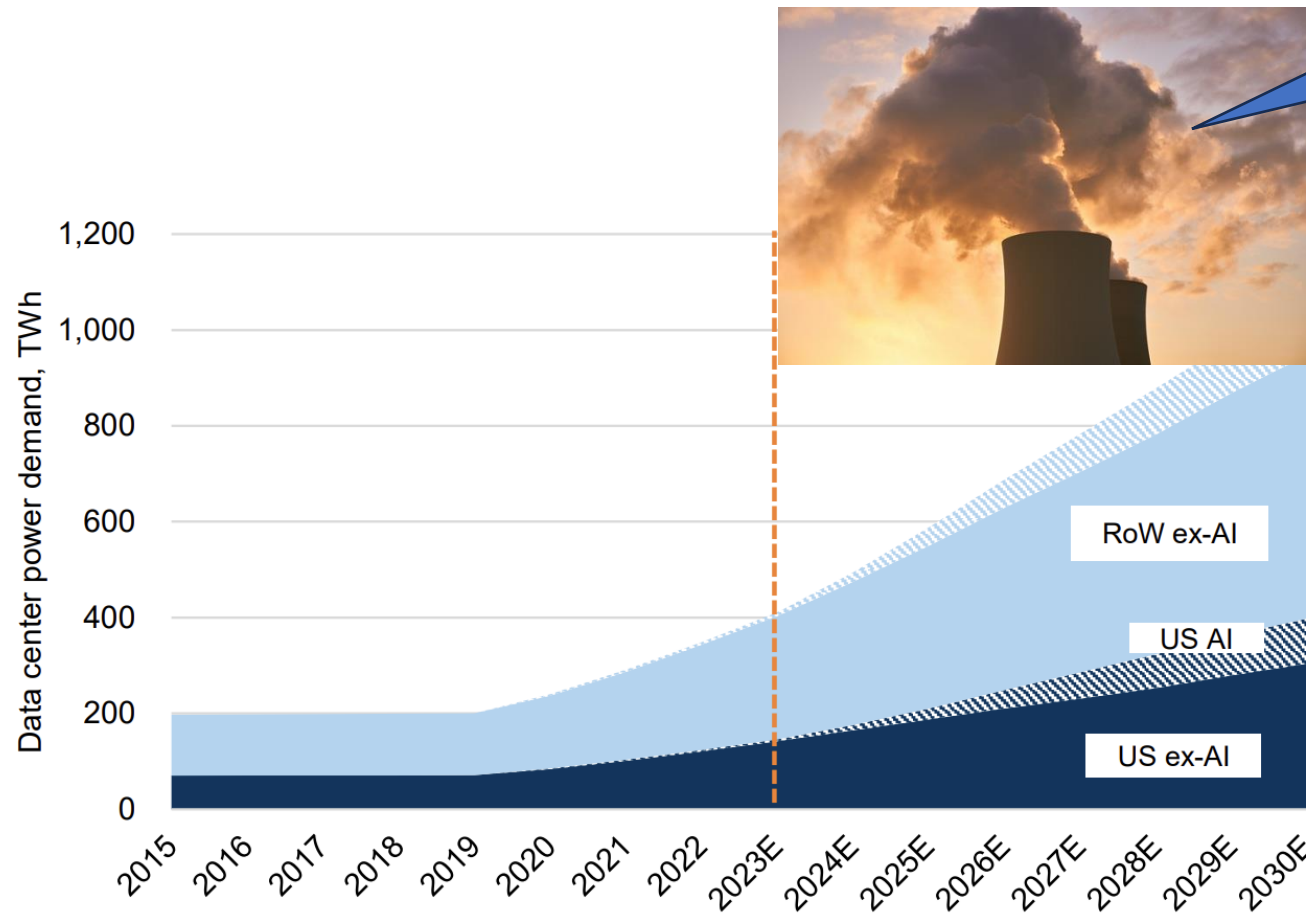
Exhibit 7: After being flat for 2015-19, we have seen data center power demand accelerate in 2021-23 and expect a 160% increase through the rest of the decade

Global data center electricity consumption, TWh; includes AI and excludes cryptocurrency



Cloud computing has a growing carbon footprint

The demand for cloud is growing
→ hard to stay green



Google's carbon emissions soar by 48% due to AI

Google's commitment to achieve net-zero by 2030 is in doubt as carbon emissions surged 48% since 2019, according to their latest environmental report.

03 July 2024 | Isabelle Nolan Finnegan

Bloomberg

• Live TV Markets Economics Industries Tech Politics Businessweek Opinion More

The AI Race: Startups to Watch AI Glossary Model Collapse Risk AI's Real Carbon Footprint Small AI Models

Green
Cleaner Tech

Microsoft's AI Push Imperils Climate Goal as Carbon Emissions Jump 30%

The company's goal to be carbon negative by 2030 is harder to reach, but President Brad Smith says the good AI can do for the world will outweigh its environmental impact.

How to make cloud computing **sustainable**?

We need to make cloud computing more **energy-efficient**.

Why is it not energy-efficient today?

Traditional cloud computing: rent VMs

- The cloud provider rents virtual machines for users to run applications

Instance Size	vCPU	Memory (GiB)	Instance Storage (GB)	Network Bandwidth (Gbps)***	EBS Bandwidth (Mbps)
m6g.medium	1	4	EBS-Only	Up to 10	Up to 4,750
m6g.large	2	8	EBS-Only	Up to 10	Up to 4,750
m6g.xlarge	4	16	EBS-Only	Up to 10	Up to 4,750
m6g.2xlarge	8	32	EBS-Only	Up to 10	Up to 4,750
m6g.4xlarge	16	64	EBS-Only	Up to 10	4,750
m6g.8xlarge	32	128	EBS-Only	12	9,000
m6g.12xlarge	48	192	EBS-Only	20	13,500
m6g.16xlarge	64	256	EBS-Only	25	19,000
m6g.metal	64	256	EBS-Only	25	19,000
m6gd.medium	1	4	1 x 59 NVMe SSD	Up to 10	Up to 4,750
m6gd.large	2	8	1 x 118 NVMe SSD	Up to 10	Up to 4,750
m6gd.xlarge	4	16	1 x 237 NVMe SSD	Up to 10	Up to 4,750
m6gd.2xlarge	8	32	1 x 474 NVMe SSD	Up to 10	Up to 4,750
m6gd.4xlarge	16	64	1 x 950 NVMe SSD	Up to 10	4,750
m6gd.8xlarge	32	128	1 x 1900 NVMe SSD	12	9,000
m6gd.12xlarge	48	192	2 x 1425 NVMe SSD	20	13,500
m6gd.16xlarge	64	256	2 x 1900 NVMe SSD	25	19,000
m6gd.metal	64	256	2 x 1900 NVMe SSD	25	19,000

Picking a good VM configuration and cluster size is hard!

Users rent more & larger VMs than they need

- Hence cloud servers are often **underutilized**
- Problem: servers consume high power even at low utilization

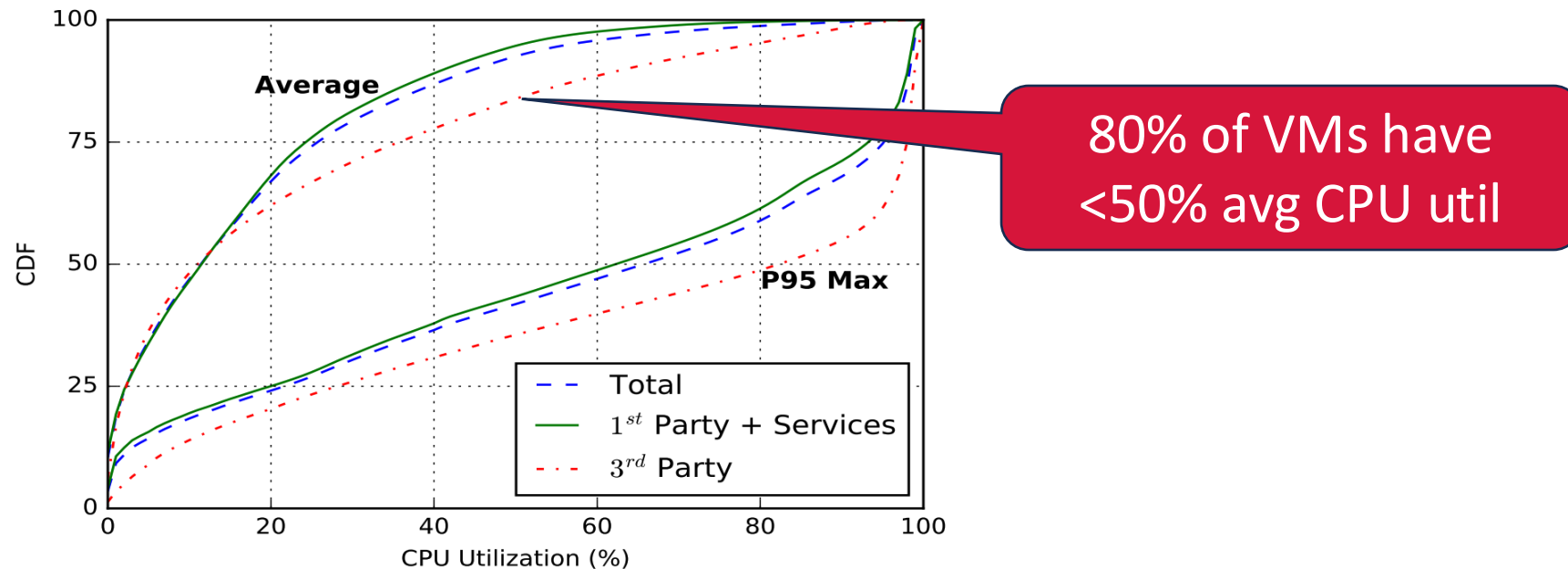


Figure 1: Average and P95 of max CPU utilizations.

Servers still consume high power at low utilization

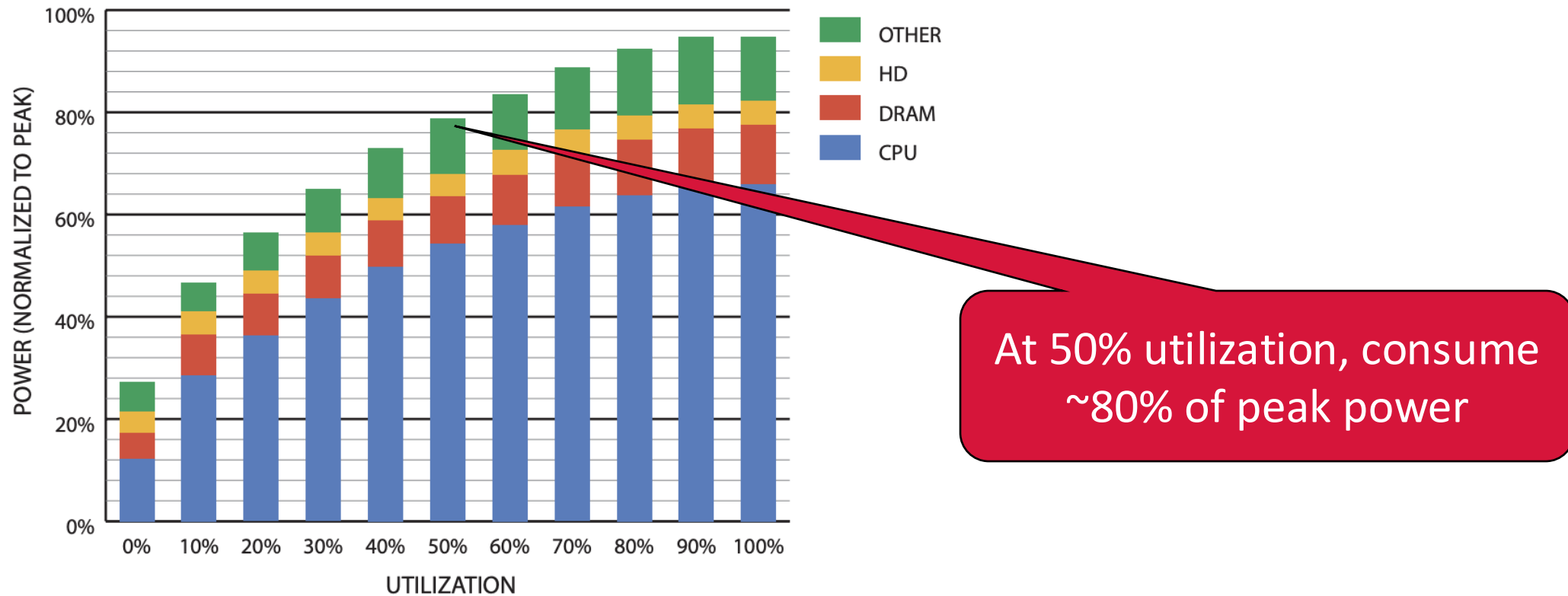
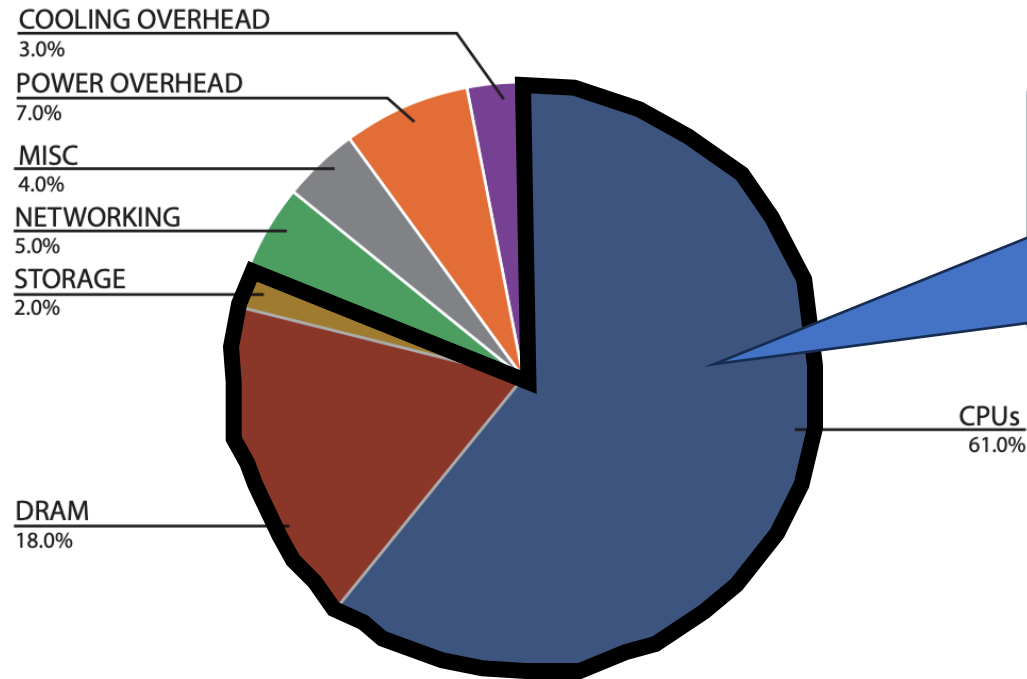


Figure 5.7: Subsystem power usage in an x86 server as the compute load varies from idle to full usage.

Servers = biggest power consumer in data centers



Need to make the most of server hardware!
→ should **maximize server utilization** so that need fewer servers overall for the job

Figure 1.8: Approximate distribution of peak power usage by hardware subsystem in a modern data center using late 2017 generation servers. The figure assumes two-socket x86 servers and 12 DIMMs per server, and an average utilization of 80%.

How to improve cloud energy efficiency?

- Ideally, should *only allocate how much CPU/GPU/memory/storage an application really needs, when it needs it.*
- What if the cloud platform could automatically allocate resources proportional to the application load?

“Serverless” computing

Users specify what tasks to execute when, the platform decides how.

Example: *“Run task X when data becomes available in storage location Y.”*



The serverless computing dream

Cloud users get:

- High level of abstraction to the cloud
- Autoscaling & billing based on load
- High throughput

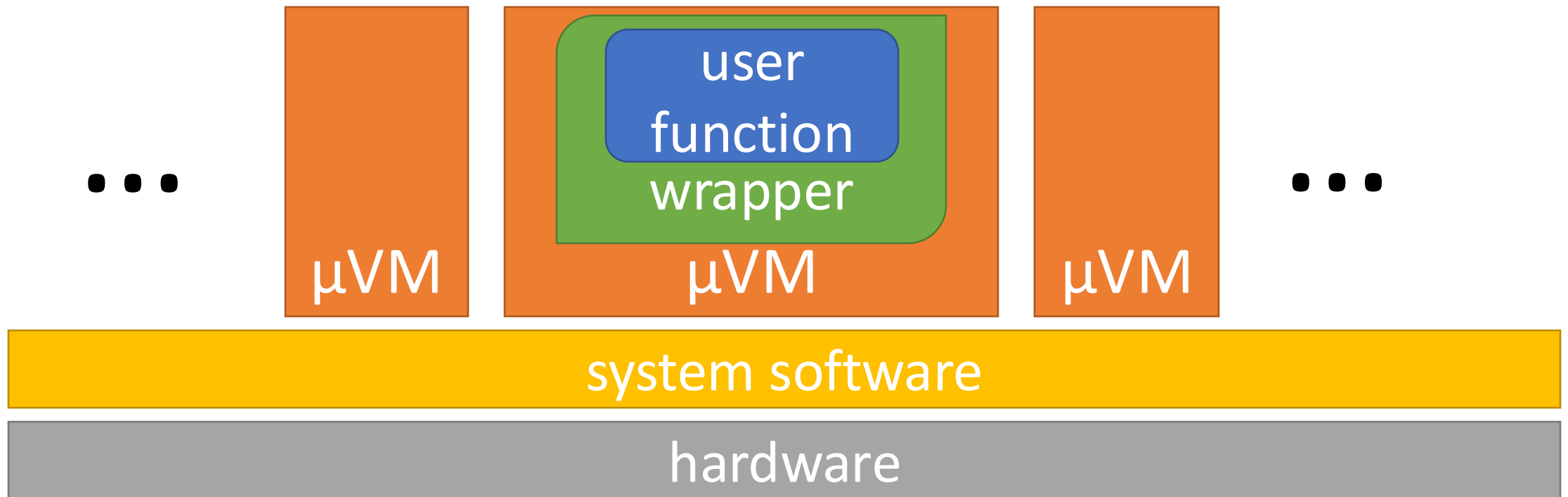
Cloud providers get:

- Opportunity to optimize energy efficiency (cost) under the hood

Ideally: make the cloud **easy to use**, while optimizing the underlying infrastructure for **performance** and **energy efficiency**

Today's serverless computing reality

- Serverless system software is still rooted in the more traditional paradigm of cloud computing, based on long-running VMs
- This leaves significant performance and energy efficiency on the table

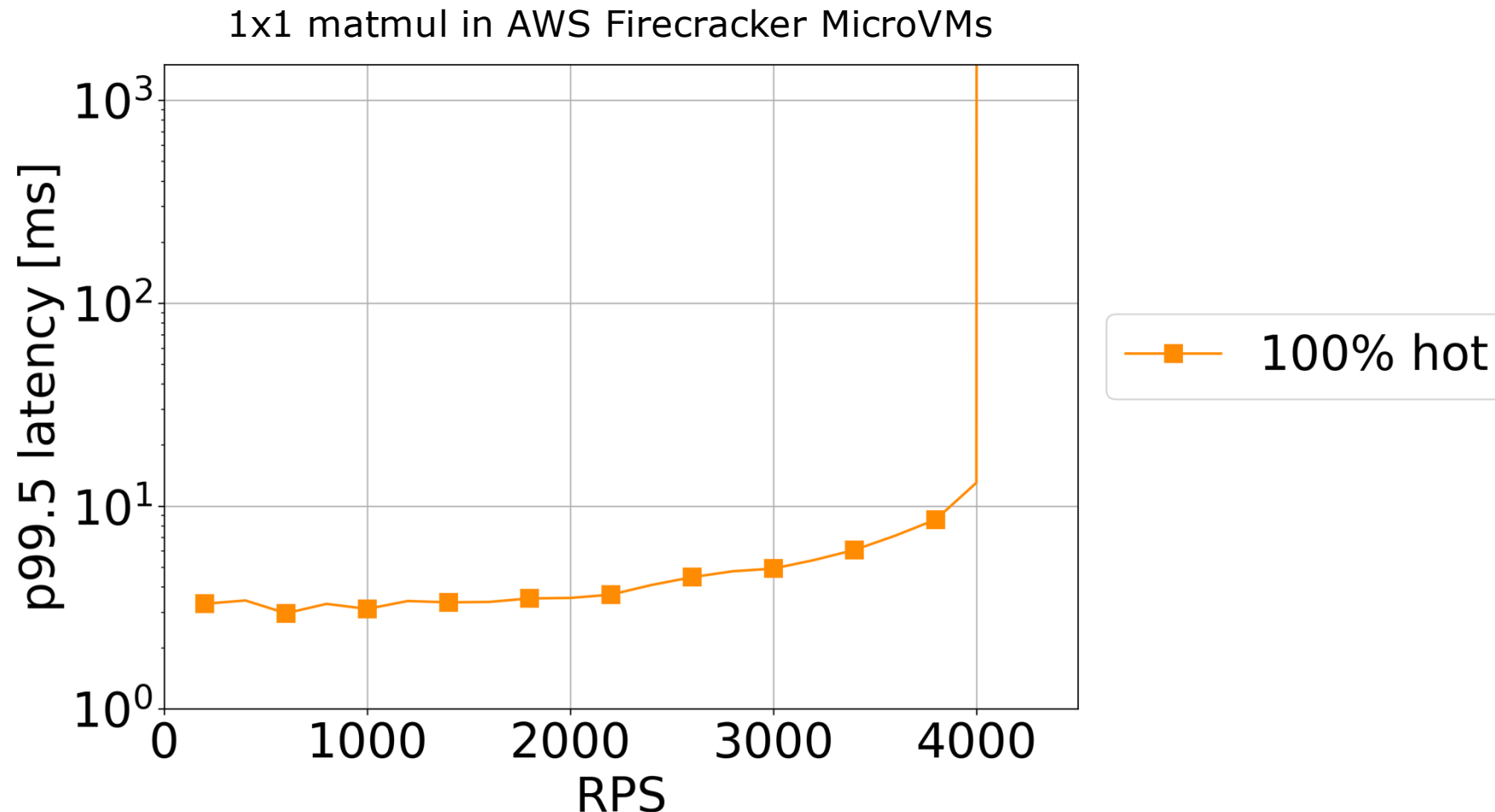


Problem 1: high tail latency due to cold starts

Booting a VM on the critical path is slow, so typically keep some “hot VMs” in memory

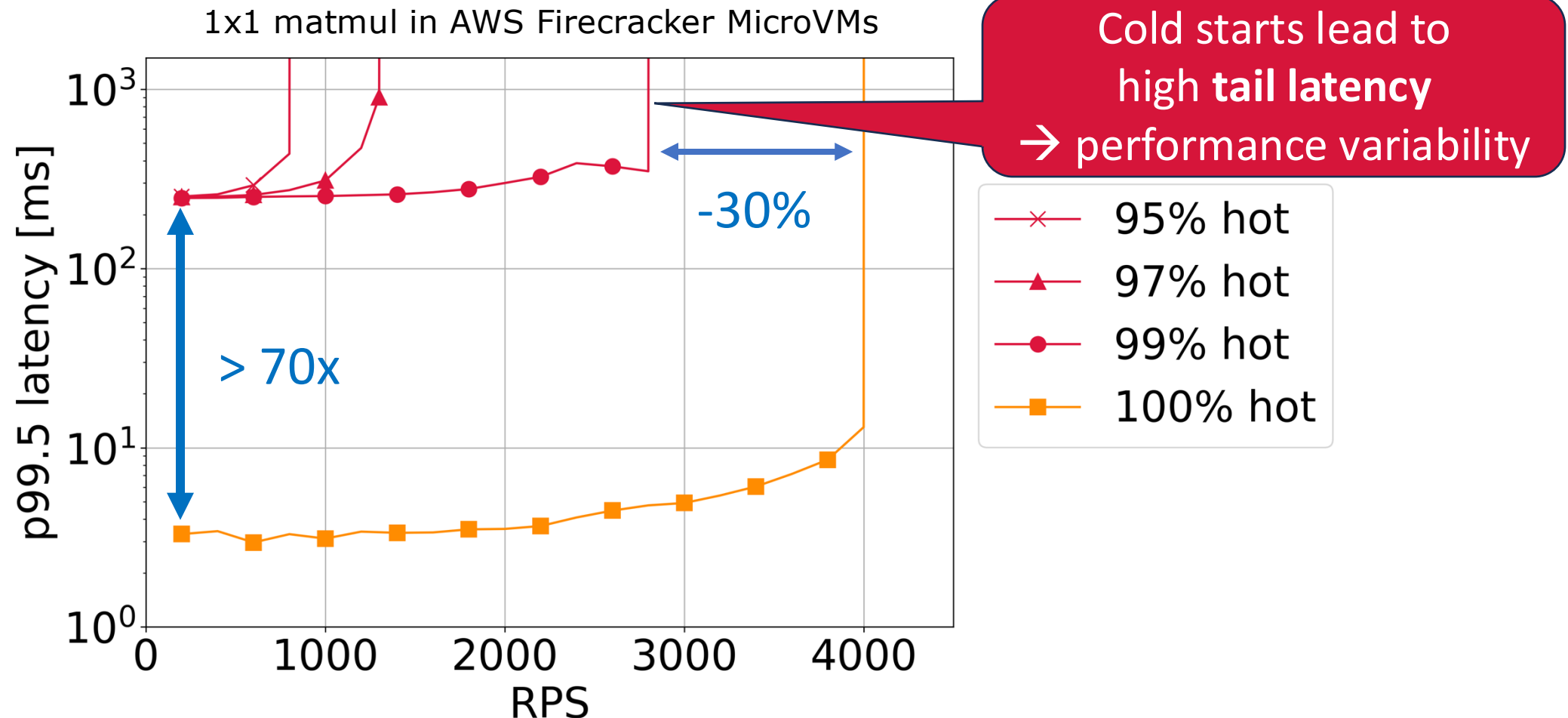
Problem 1: high tail latency due to cold starts

Booting a VM on the critical path is slow, so typically keep some “hot VMs” in memory



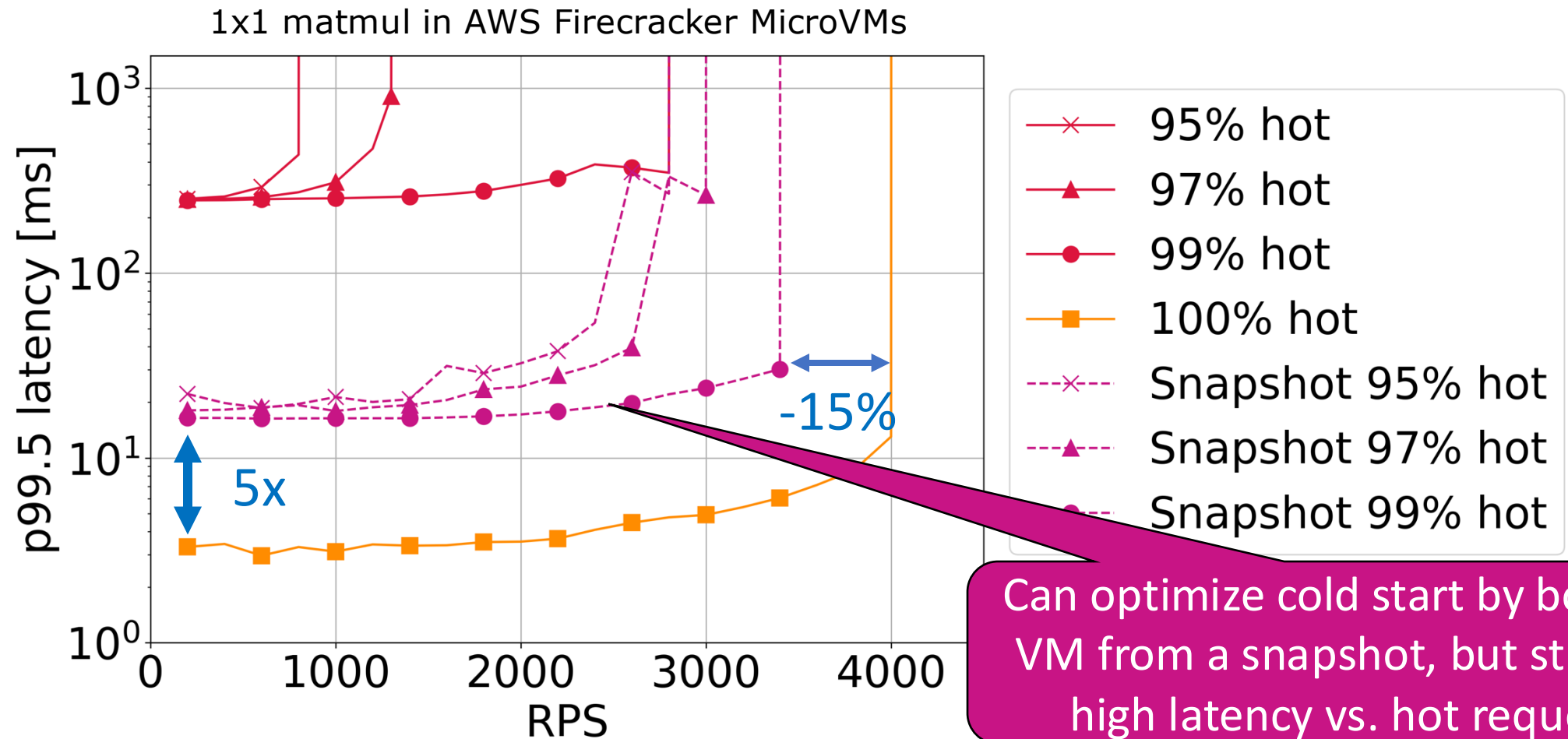
Problem 1: high tail latency due to cold starts

Booting a VM on the critical path is slow, so typically keep some “hot VMs” in memory



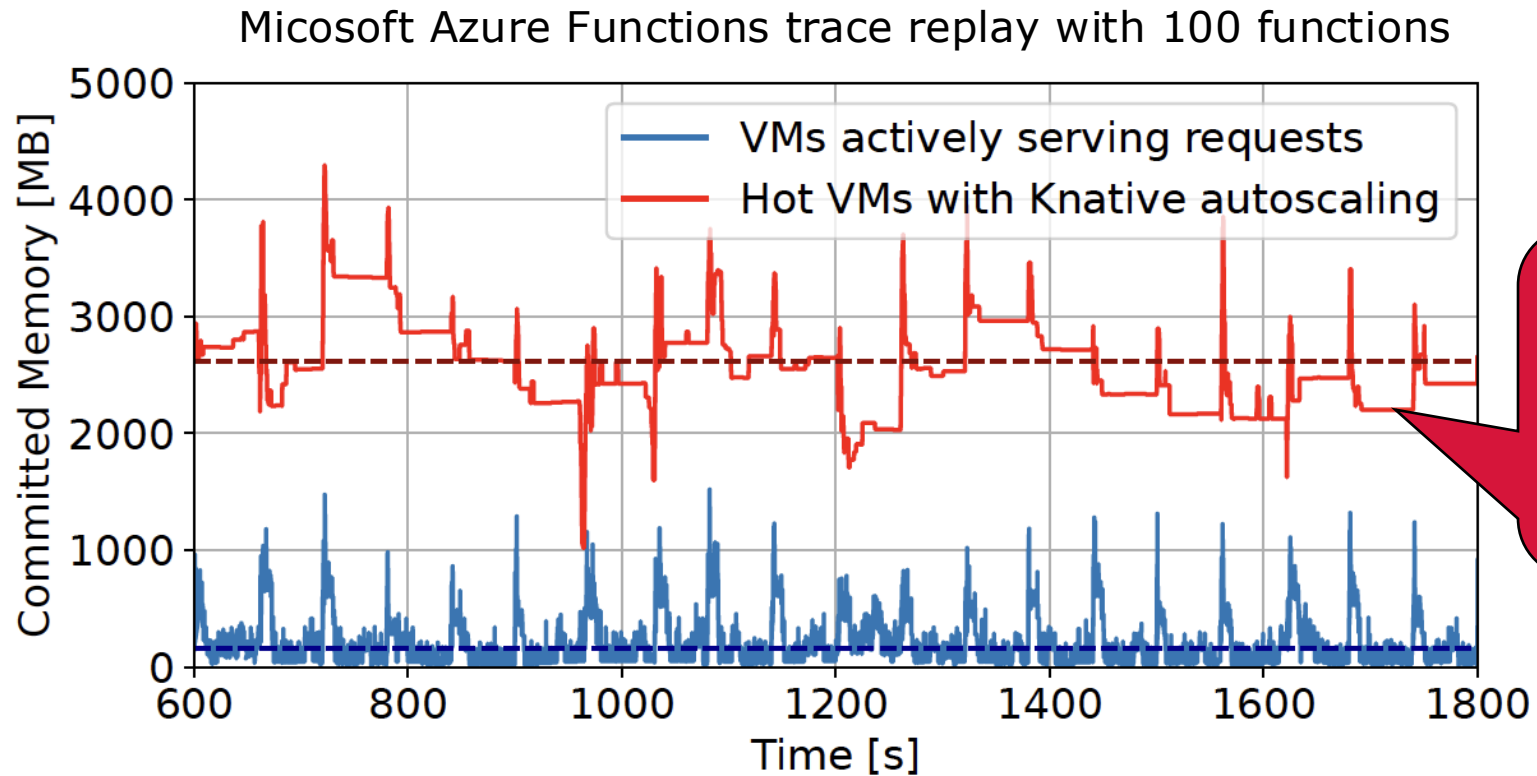
Problem 1: high tail latency due to cold starts

Booting a VM on the critical path is slow, so typically keep some “hot VMs” in memory



Problem 2: high memory overhead of hot VMs

The cloud platform needs to commit DRAM for each hot VM → memory is expensive!

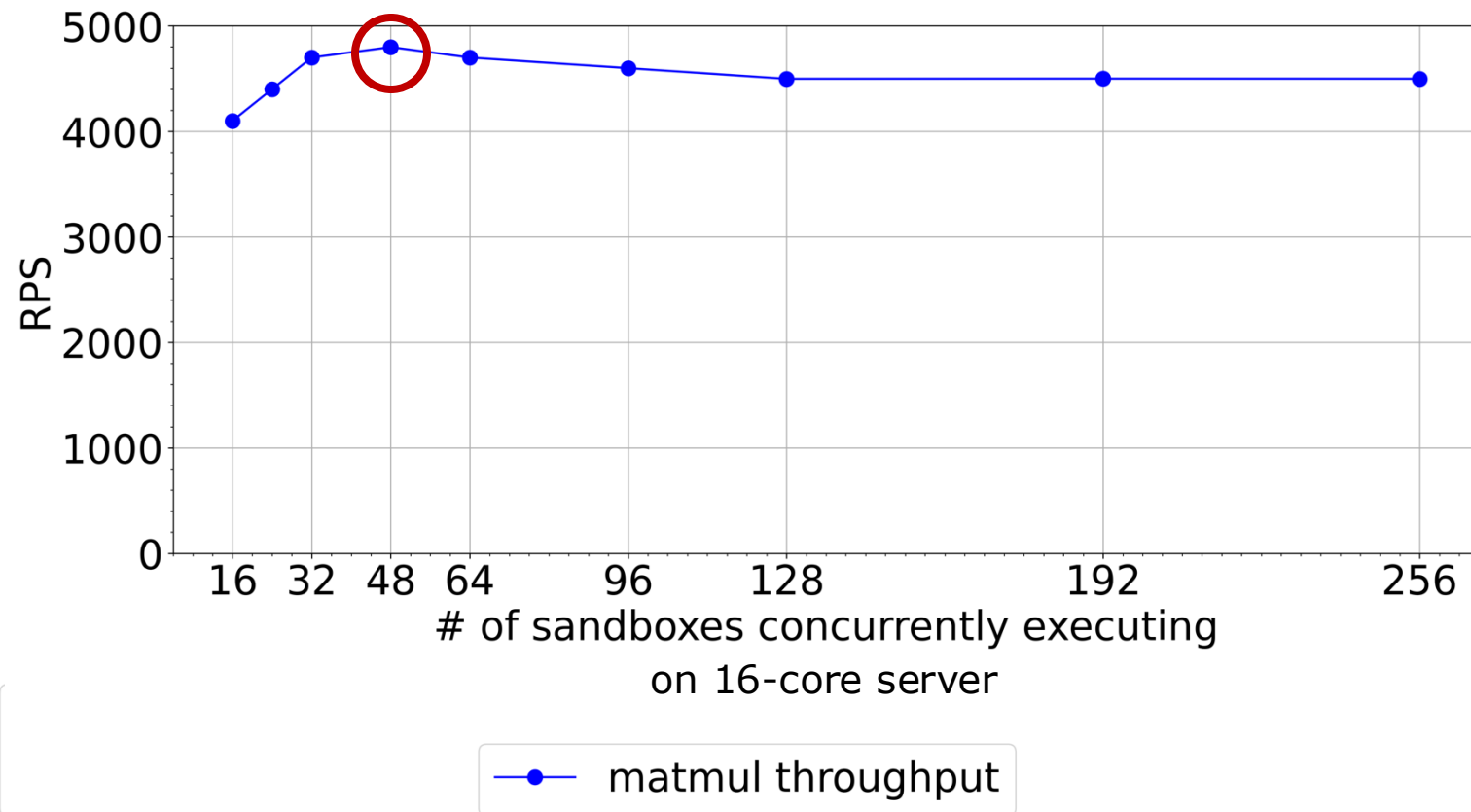


State-of-the-art autoscaling achieves 97% hot starts, but needs **16x higher memory** on avg for the hot VMs!

Problem 3: hard to optimize CPU allocation

How much to overcommit CPU cores to process many concurrent requests?

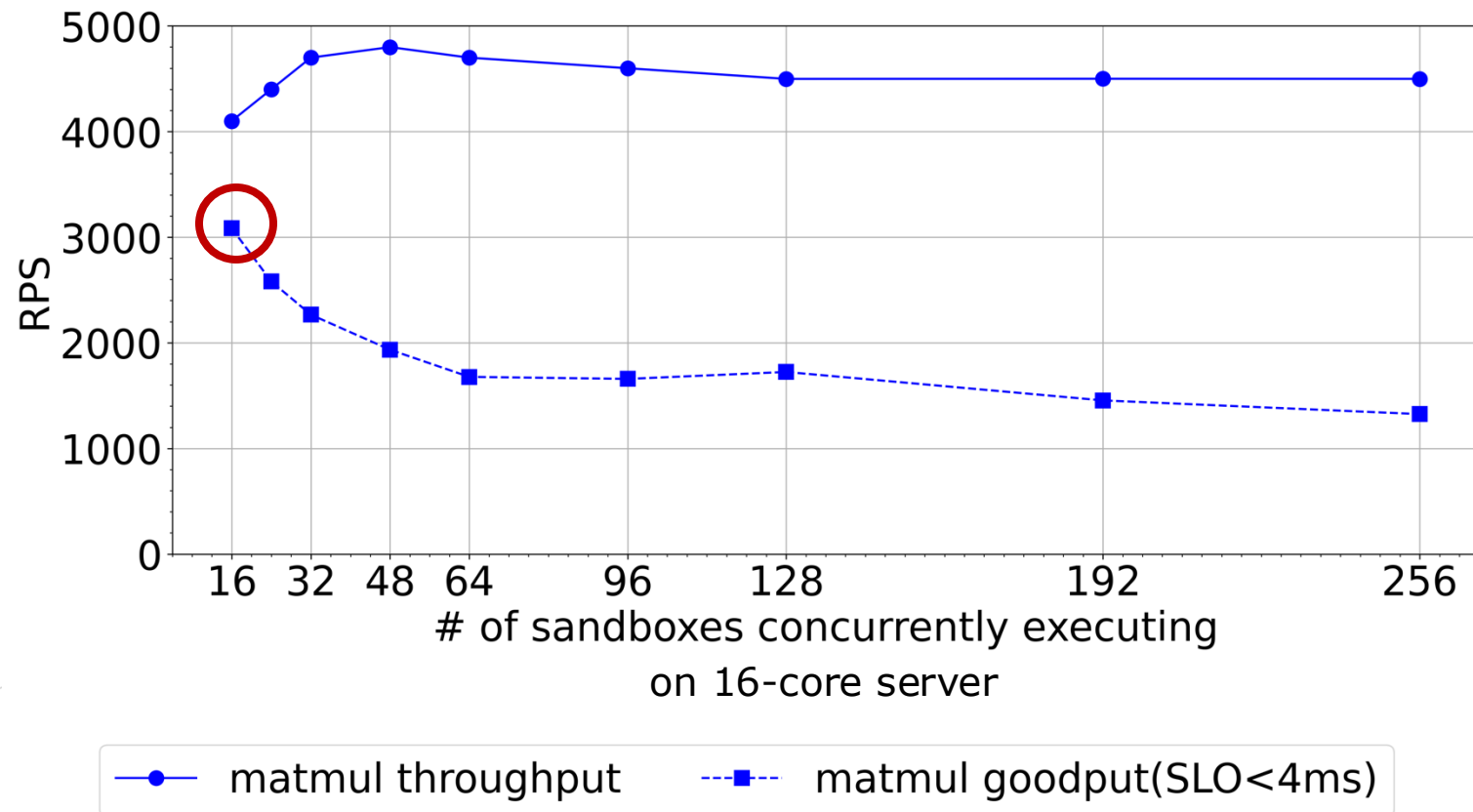
→ depends on the app's compute vs. I/O intensity, but the platform as no insight!



Problem 3: hard to optimize CPU allocation

How much to overcommit CPU cores to process many concurrent requests?

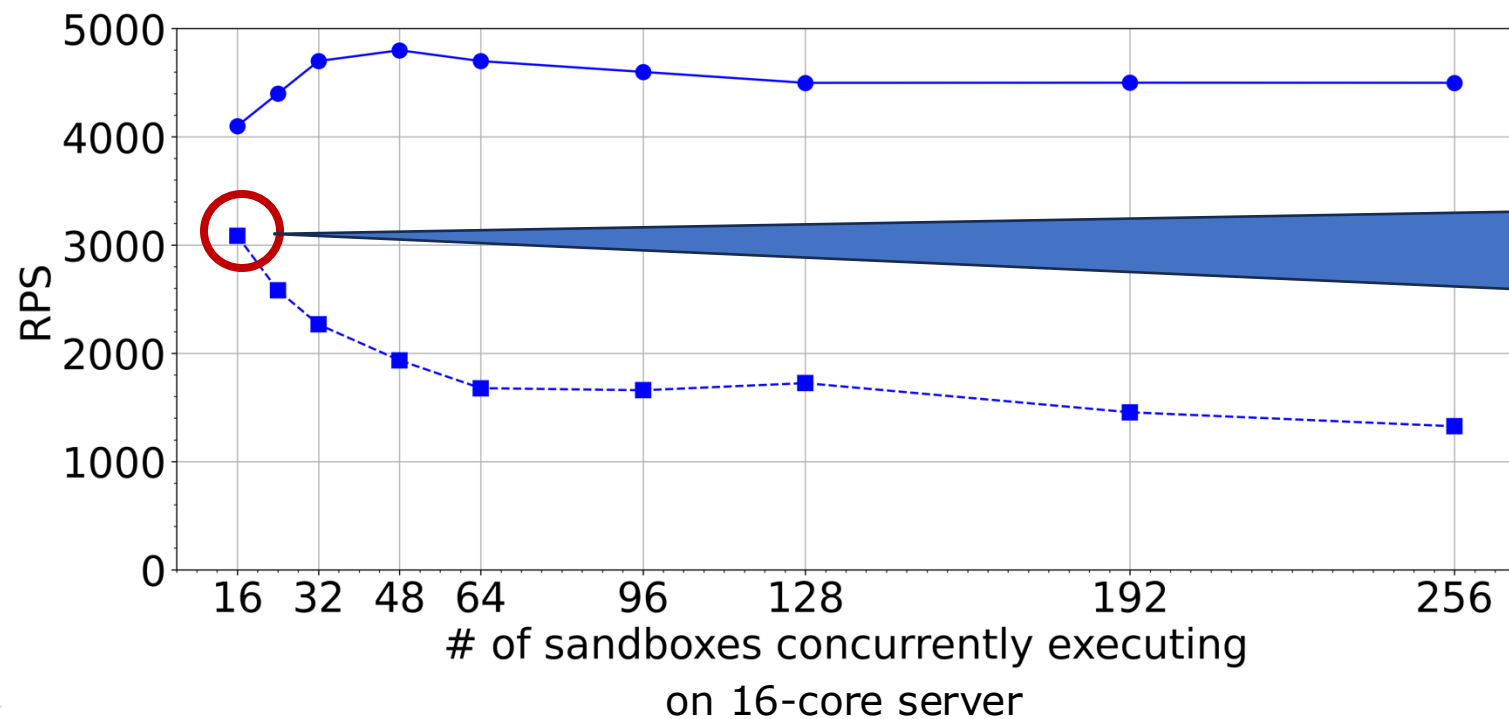
→ depends on the app's compute vs. I/O intensity, but the platform as no insight!



Problem 3: hard to optimize CPU allocation

How much to overcommit CPU cores to process many concurrent requests?

→ depends on the app's compute vs. I/O intensity, but the platform as no insight!



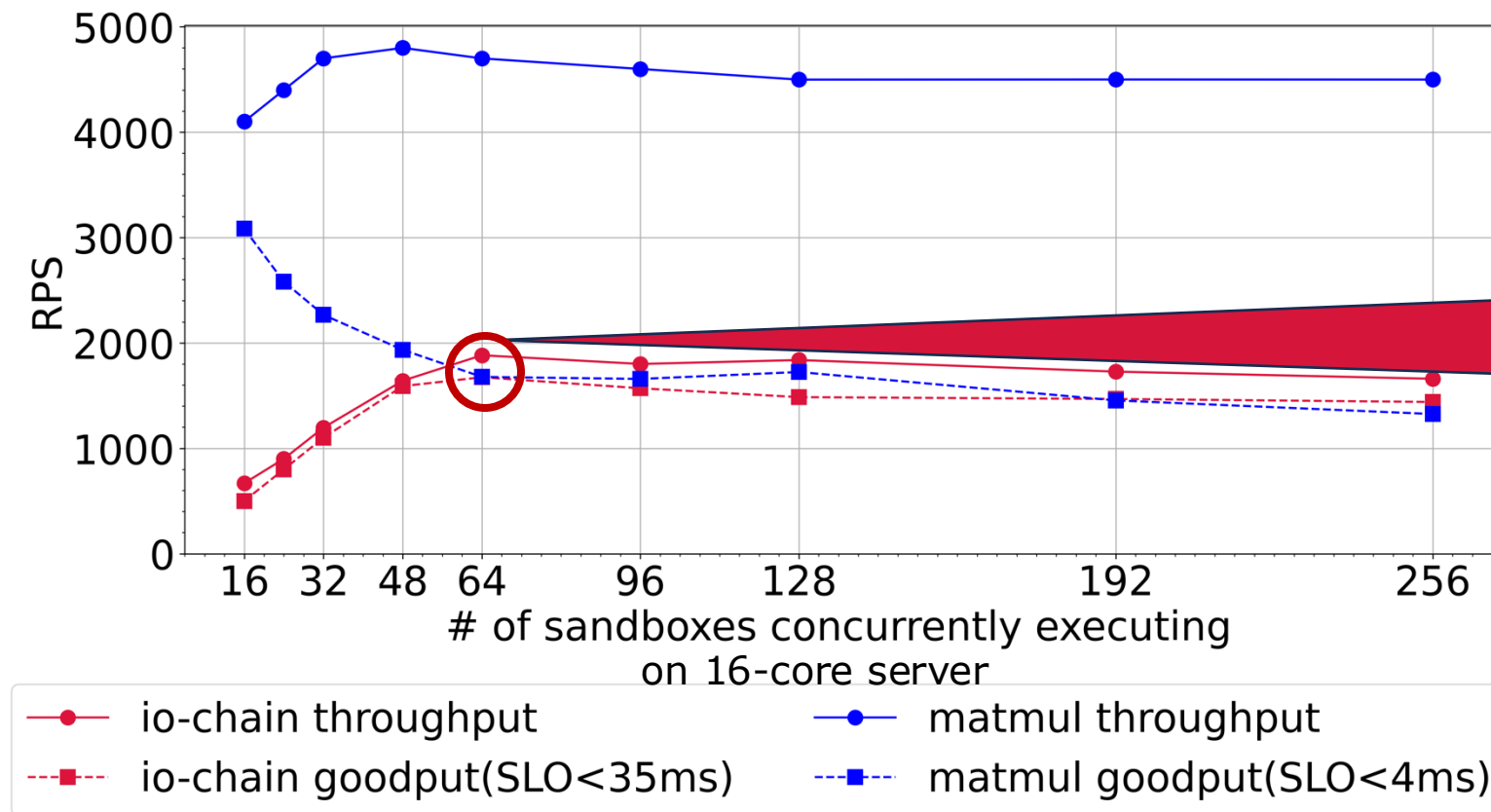
Overcommitting CPU for a compute-intensive app increases latency

—●— matmul throughput - -■- - matmul goodput(SLO<4ms)

Problem 3: hard to optimize CPU allocation

How much to overcommit CPU cores to process many concurrent requests?

→ depends on the app's compute vs. I/O intensity, but the platform has no insight!



For this I/O-intensive app,
should overcommit CPU
cores by 4x

The fundamental problem

Poor interface between cloud platform & user applications:

- The platform treats each application as a black box; it has no insight about the compute vs. I/O intensity of each application
 - Hard to optimize CPU and memory allocations → high cost & energy usage
- To support general applications, the platform exposes a broad interface (i.e., syscalls) to untrusted user code, which leads to a large attack surface
 - Requires heavy isolation mechanisms like VMs for security → poor performance

Outline

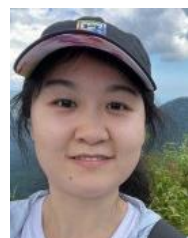
- Part 1: Dandelion
 - **Programing model** with a declarative interface between cloud app & platform
 - **Execution system** that runs on each server, co-designed with the programming model for high performance and energy efficiency
- Part 2: Dirigent
 - **Cluster orchestration system** that manages applications across servers



Tom Kuchler



Pinghe Li



Yazhuo Zhang



Leon Thomm



Lazar Cvetković



Andrea Lattuada

Dandelion

Key idea: new cloud-native programming model that strictly separates **computation** and **communication**

compute functions

- untrusted user code
- pure functions
- platform prepares the function's inputs and extracts its outputs

communication functions

- trusted platform code
- allowed to interact with system software or hardware

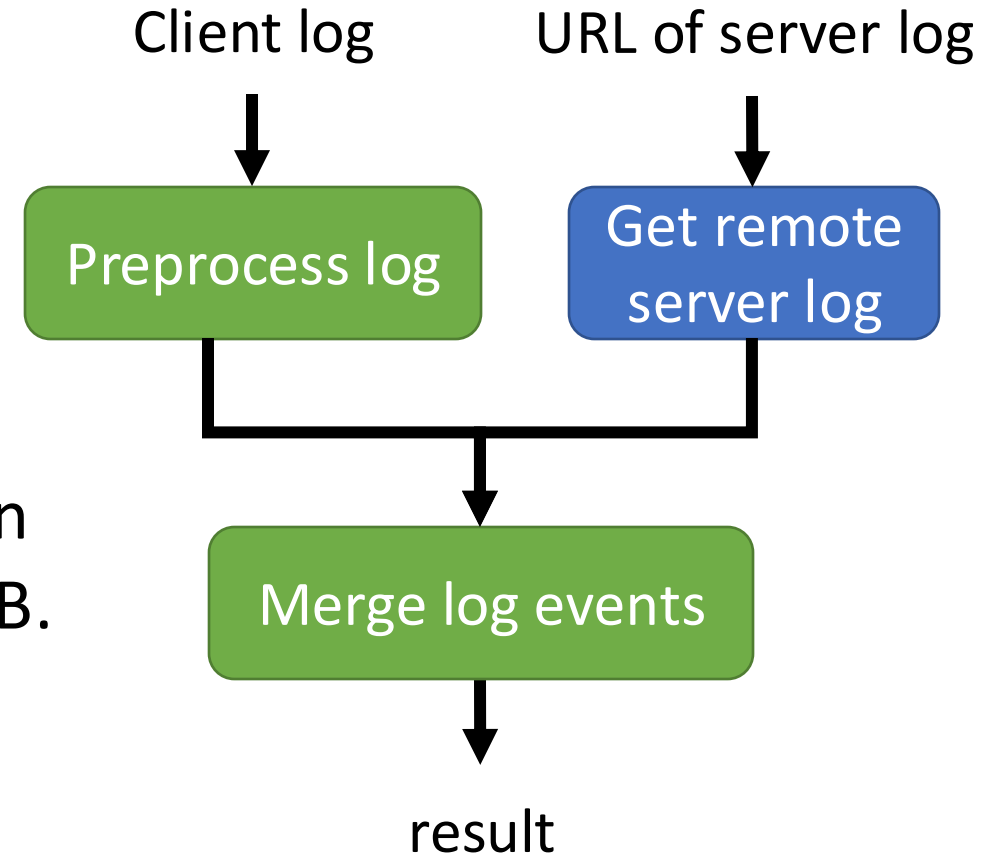
Dandelion Function Compositions

Users develop apps by composing:

untrusted compute functions &
trusted communication functions

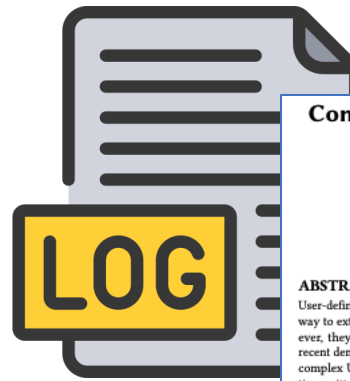
Communication functions enable interaction with external services, e.g. cloud storage, DB.

Compute functions are not allowed to issue syscalls.



Target applications

- Distributed log processing
- Data processing applications
 - Secure, elastic UDF execution
 - Database "cracking" tasks: transform source data (transcode, sort, chunk) to improve expected query performance
 - Query execution with elastic per-query resource scaling
- Agentic AI inference systems



Containerized Execution of UDFs: An Experimental Evaluation

Karla Saur
Microsoft
karla.saur@microsoft.com

Konstantinos Karanasos[†]
Meta
kkaranasos@fb.com

Tara Mirmira^{*}
University of California, San Diego
tmirmira@eng.ucsd.edu

Jesús Camacho-Rodríguez
Microsoft
jesusca@microsoft.com

ABSTRACT

User-defined functions (UDFs) are a way to extend the capabilities of a database. However, they are recent demand for complex UDFs written in dependencies, resource isolation, UDFs were designed that enable elastic and usability issues at all stages of code in their execution.

Our experience with UDF execution that there are the method of that binary-based more than 2.4x a newer generation latency suboptimal in can utilize simple data sorting) using server input data based on d to improve latency an act as a middleware transformations, where tency/cost budgets. F these transformations.

PVLDB Refere
Karla Saur, Tara Camacho-Rodríguez. Evaluation. PVLDB 10:14778/3551.

PVLDB Artifact
The source code is available at <https://github.com>.

^{*}Work performed at Microsoft. This work is licensed under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

[†]Work performed at Meta. This work is licensed under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

^{*}Currently at Apple. This paper is published under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

PVLDB Reference For
Dimitrios Koutsoukos, Ingo Müller, Renato Marroquín. Modular Heterogeneous Distributed

^{*}The work of this author was supported by the National Science Foundation (NSF) under grant number 1014778/3551. This work is licensed under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

Adaptive data transformations for QaaS

Dimitrios Koutsoukos^{*}
ETH Zurich
Switzerland
koutsoukos.dimitr@gmail.com

Ingo Müller
Google
Switzerland

Renato Marroquín
Oracle Inc.
Switzerland
renato.marroquin@oracle.com

Ana Klimovic
ETH Zurich
Switzerland

ABSTRACT

In today's data-driven world, the demand for semi-structured data is growing. Query-as-a-Service (QaaS) allows the user to query the data without the need for dedicated database systems. However, traditional database systems are not designed for this use case, leading to high latency and cost. In this paper, we present Adaptive data transformations for QaaS, a framework for transforming source data into a format that can be processed by a distributed database system. Our framework uses a combination of static and dynamic transformations to optimize the data for the target system. We evaluate our framework on a real-world dataset and show that it can reduce latency and cost by up to 10x.

PVLDB Refere
Karla Saur, Tara Camacho-Rodríguez. Evaluation. PVLDB 10:14778/3551.

PVLDB Artifact
The source code is available at <https://github.com>.

^{*}Work performed at Microsoft. This work is licensed under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

[†]Work performed at Meta. This work is licensed under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

^{*}Currently at Apple. This paper is published under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

PVLDB Reference For
Dimitrios Koutsoukos, Ingo Müller, Renato Marroquín. Modular Heterogeneous Distributed

^{*}The work of this author was supported by the National Science Foundation (NSF) under grant number 1014778/3551. This work is licensed under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

Modularis: Modular Relational Analytics over Heterogeneous Distributed Platforms

Dimitrios Koutsoukos
ETH Zurich, Switzerland
dkoutsou@inf.ethz.ch

Ingo Müller
ETH Zurich, Switzerland
ingo.mueller@inf.ethz.ch

Renato Marroquín^{*}
Oracle Inc., Zurich, Switzerland
renato.marroquin@oracle.com

Ana Klimovic
ETH Zurich, Switzerland
aklimovic@ethz.ch

Gustavo Alonso
ETH Zurich, Switzerland
alonso@inf.ethz.ch

ABSTRACT

The enormous quantity of data produced every day together with advances in data analytics has led to a proliferation of data management and analysis systems. Typically, these systems are built around highly specialized monolithic operators optimized for the underlying hardware. While effective in the short term, such an approach makes the operators cumbersome to port and adapt, which is increasingly required due to the speed at which algorithms and hardware evolve. To address this limitation, we present *Modularis*, an execution layer for data analytics based on *sub-operators*, i.e., composable building blocks resembling traditional database operators but at a finer granularity. To demonstrate the feasibility and advantages of our approach, we use *Modularis* to build a distributed query processing system supporting relational queries running on an RDMA cluster, a serverless cloud platform, and a smart storage engine. *Modularis* requires minimal code changes to execute queries across these three diverse hardware platforms, showing that the sub-operator approach reduces the amount and complexity of the code to maintain. In fact, changes in the platform affect only those sub-operators that depend on the underlying hardware (in our use cases, mainly the sub-operator for RDMA clusters). We show the end-to-end performance of our framework for a cluster database (SingleStore, Athena, BigQuery) proving that the design can be achieved. We compare *Modularis* with existing systems, showing that it is easily extensible by queries, all of which are supported by the underlying hardware.

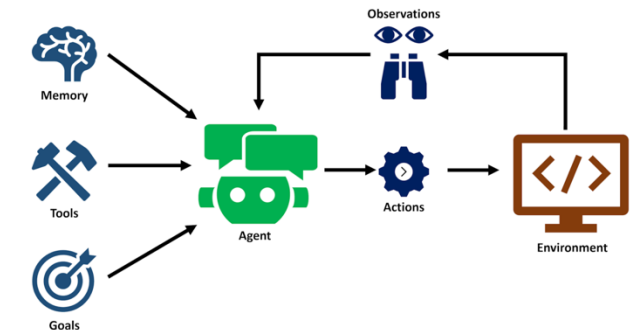
PVLDB Reference For
Dimitrios Koutsoukos, Ingo Müller, Renato Marroquín. Modular Heterogeneous Distributed

^{*}The work of this author was supported by the National Science Foundation (NSF) under grant number 1014778/3551. This work is licensed under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

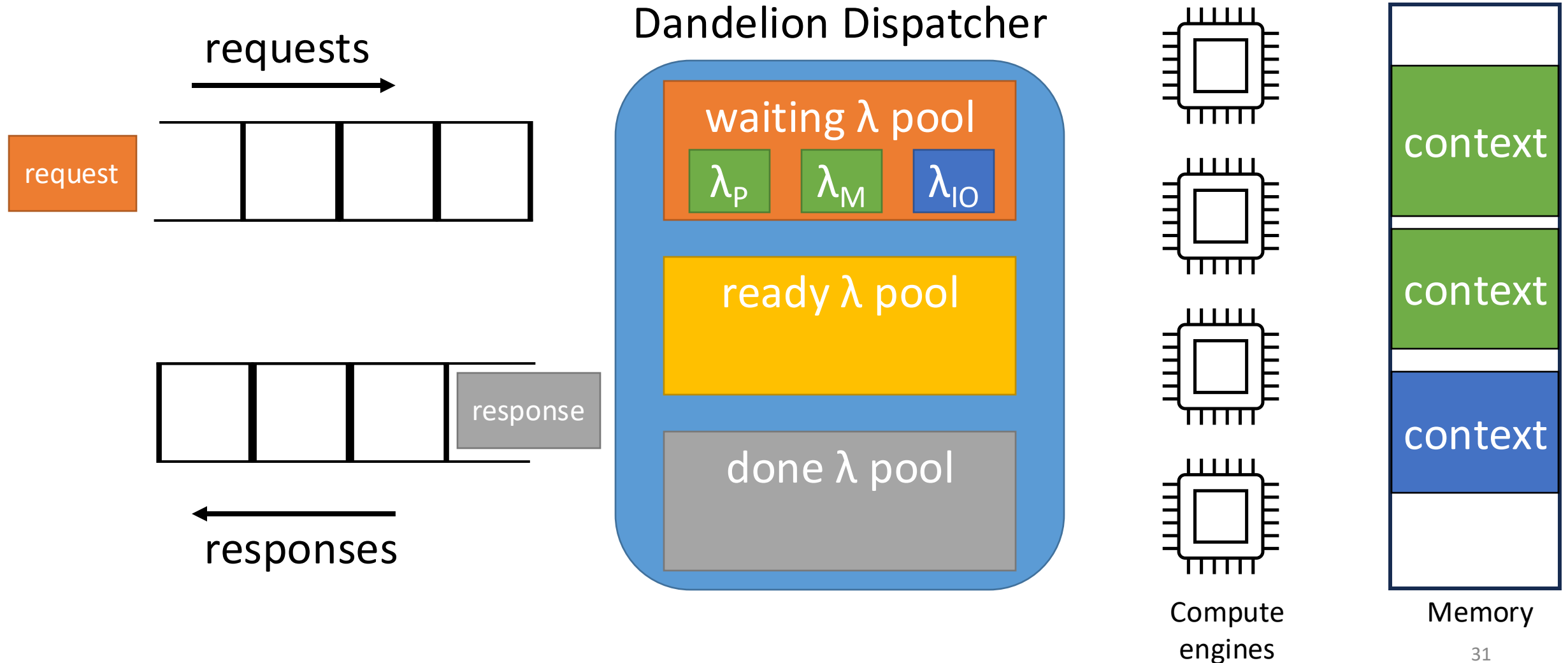
^{*}The work of this author was supported by the National Science Foundation (NSF) under grant number 1014778/3551. This work is licensed under the Creative Commons Attribution 4.0 International License. Visit <https://creativecommons.org/licenses/by/4.0/> for details. For an extended version, see the full paper at the VLDB Proceedings of the VLDB 10:14778/3551.

1 INTRODUCTION

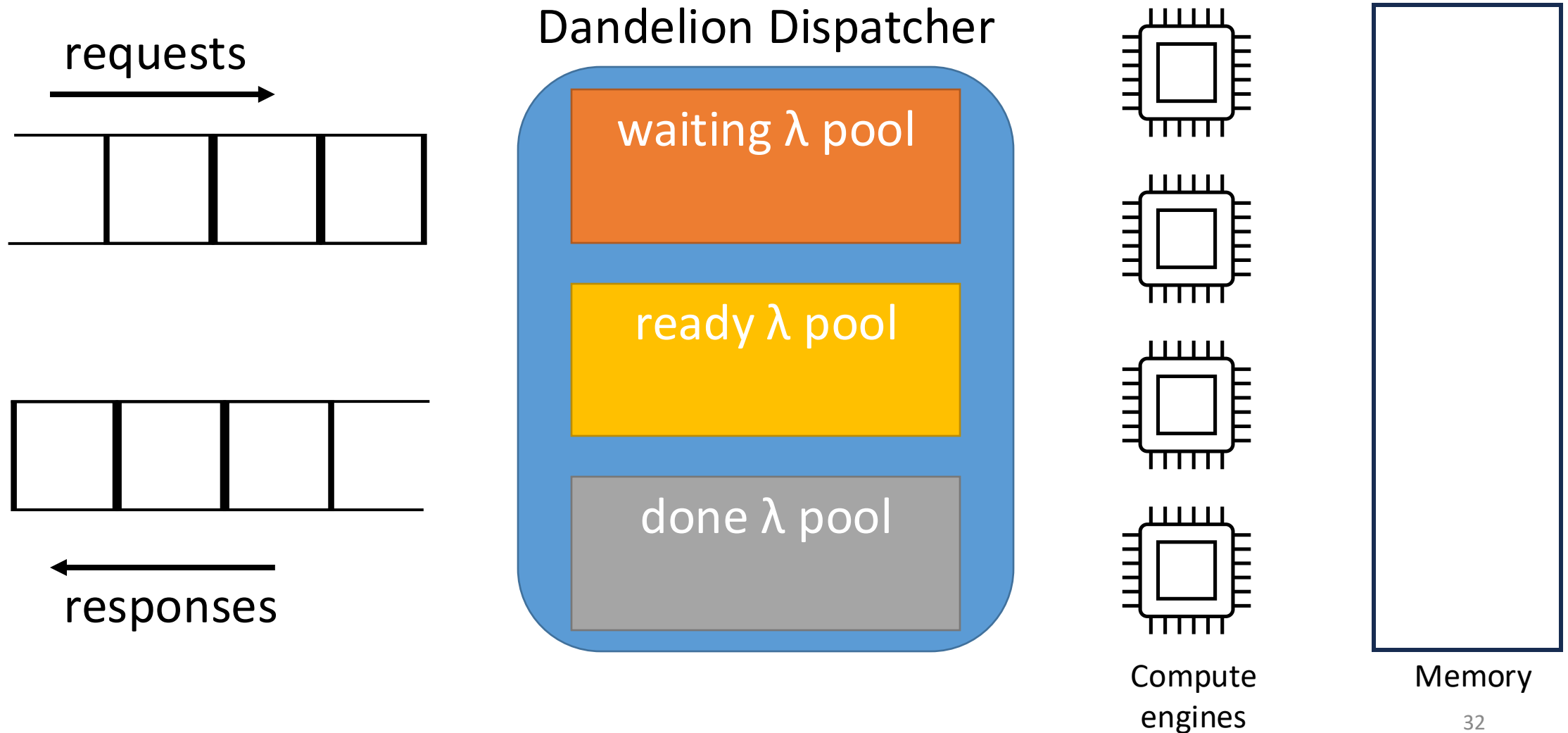
The growing popularity of machine learning applications and the increasing amount of data that analytics applications must process have had a substantial influence on the way systems are designed and optimized. There is a constant stream of specialized accelerators (TPUs, GPUs, FPGAs, smart NICs, smart storage, near memory processing) and platforms (large appliances, InfiniBand clusters, serverless, cloud instances, data centers) that force a continuous redesign of data processing engines—often leading to new engines—simply to exploit the capabilities of new hardware [16]. Often, to gain performance, developers design monolithic operators that are highly tailored to the underlying hardware [11, 13, 14, 52, 56]. However, as the algorithms and platforms evolve quickly, it becomes very difficult to reuse these operators in newer versions of the system. Examples abound: For instance, a join optimized for multi-core machines [11] requires fundamental changes to run on Remote Direct Memory Access (RDMA) [13, 14] due to the different communication schemes between NUMA nodes and the network. As another example, although FPGAs are not competitive with multi-core machines for full joins, they can significantly accelerate



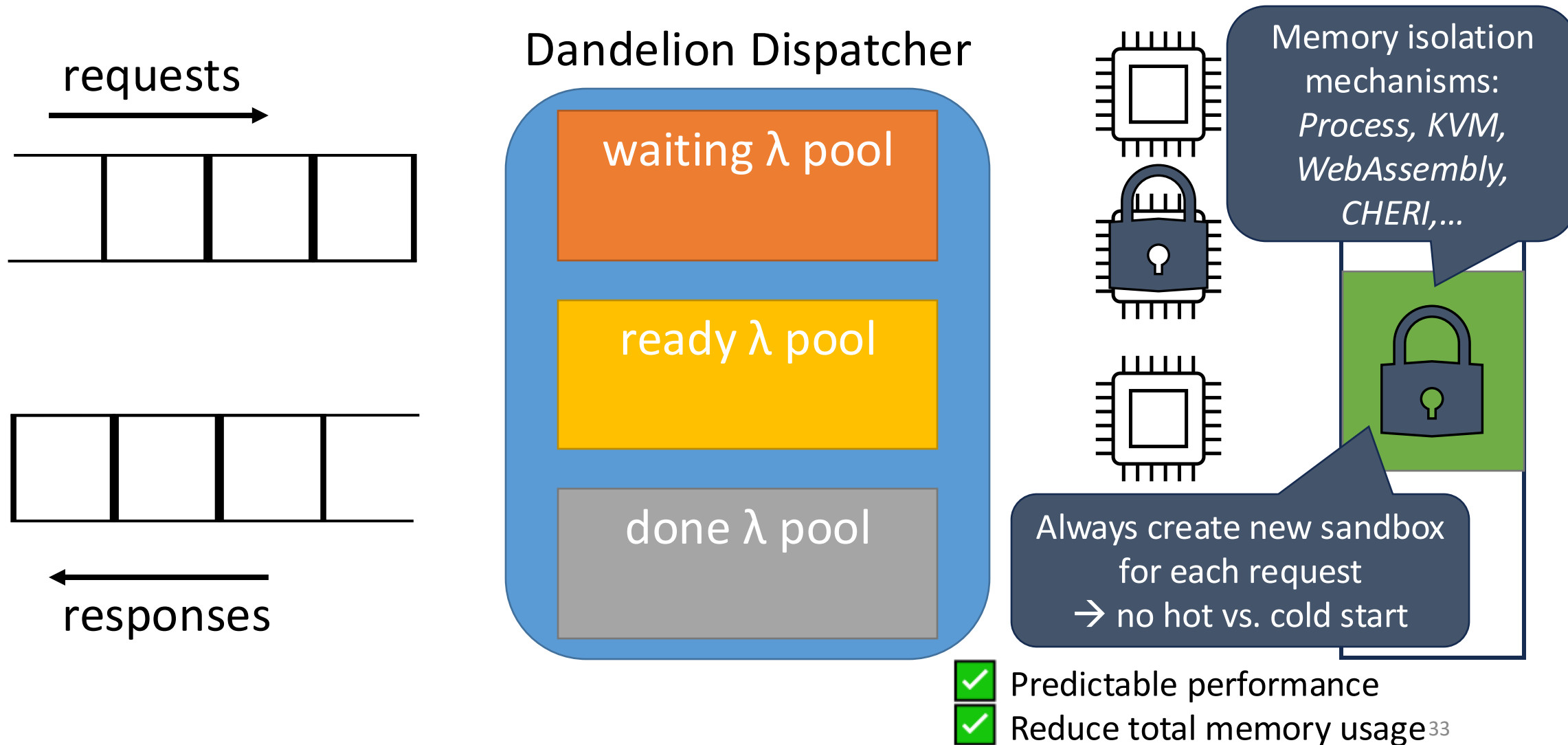
Function execution system



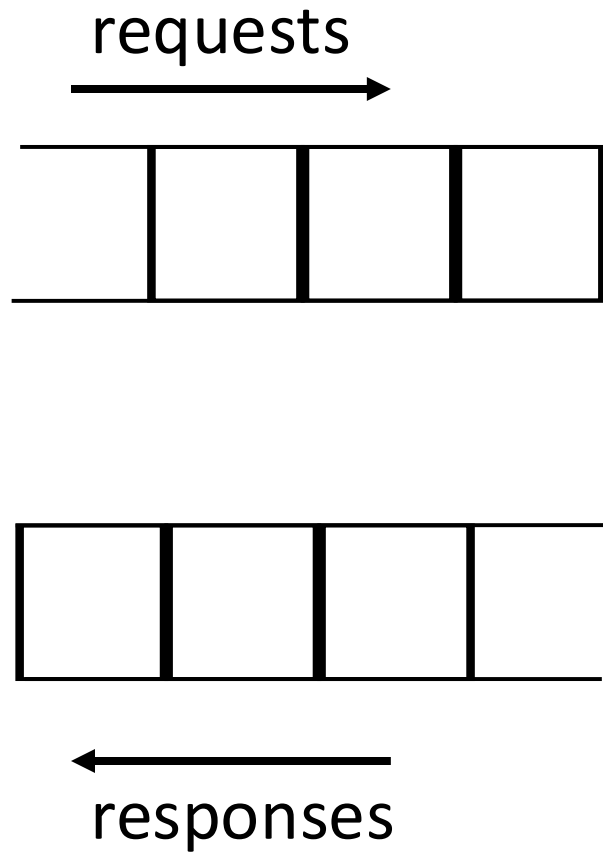
Why Dandelion?



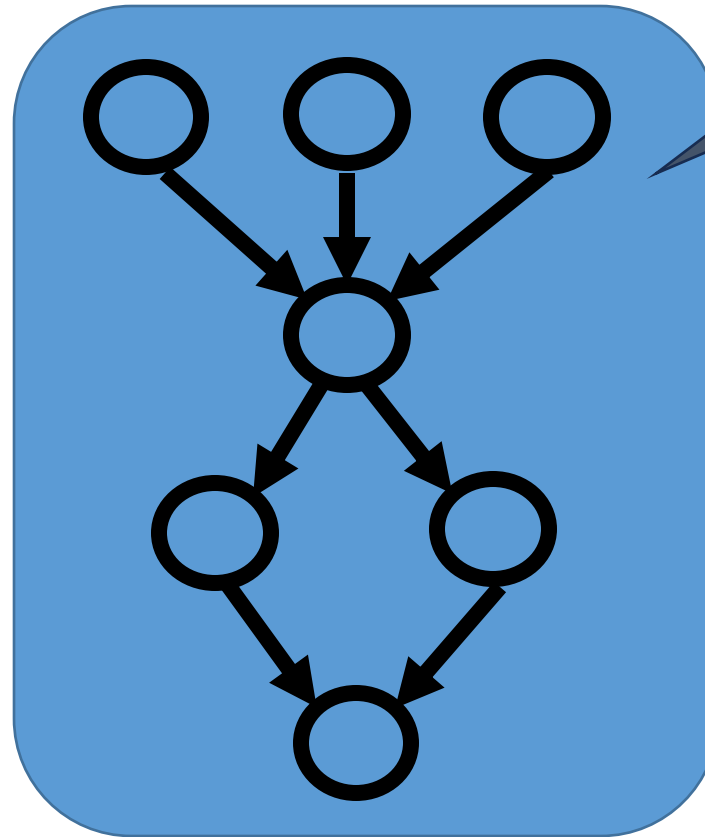
1. Fast execution with lightweight isolation



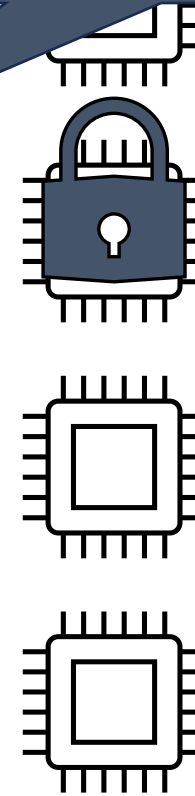
2. Dataflow aware scheduling



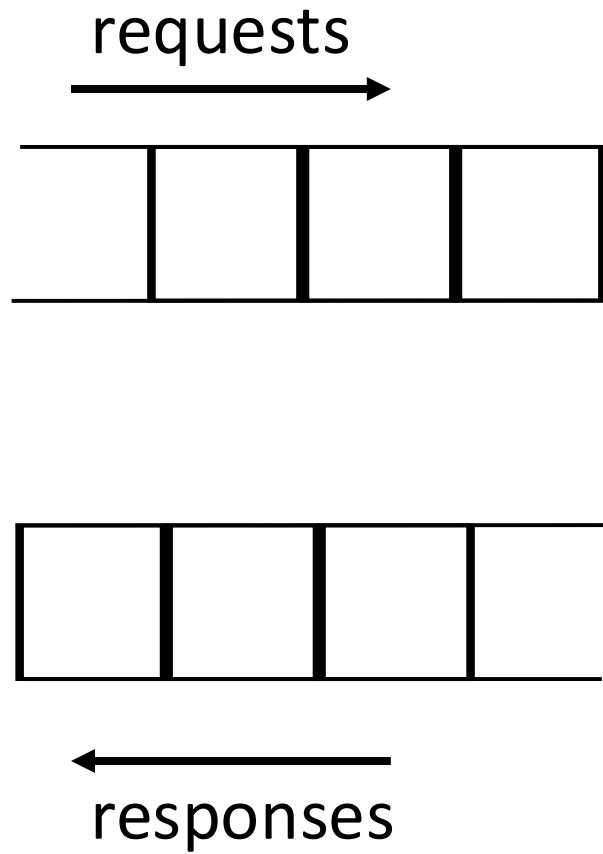
Dandelion Dispatcher



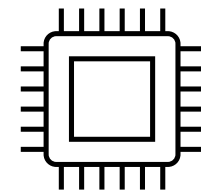
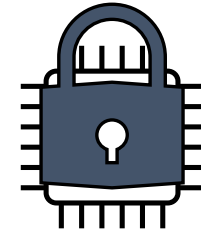
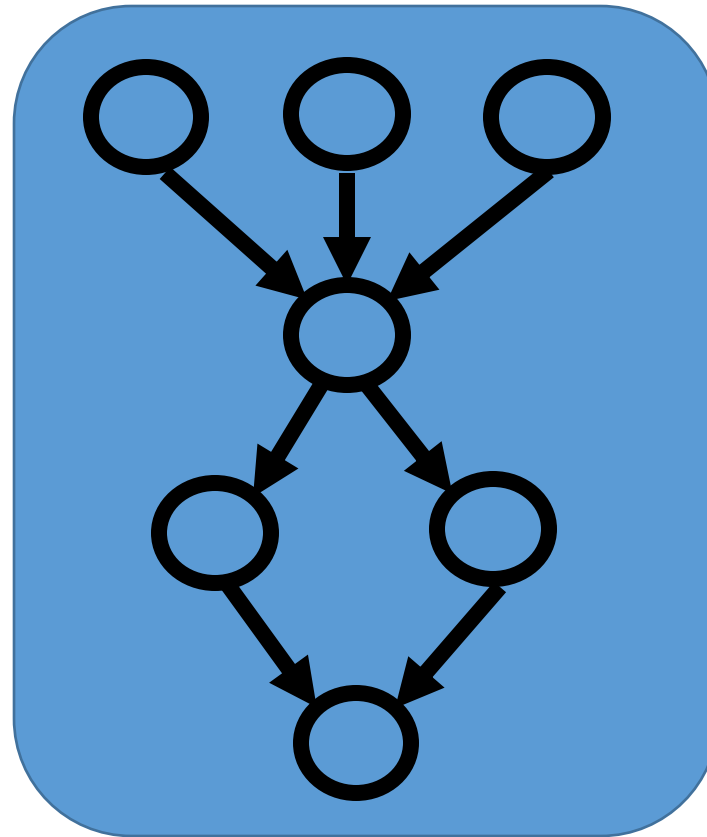
- Run compute tasks to completion
- Only schedule a task when *all* inputs ready



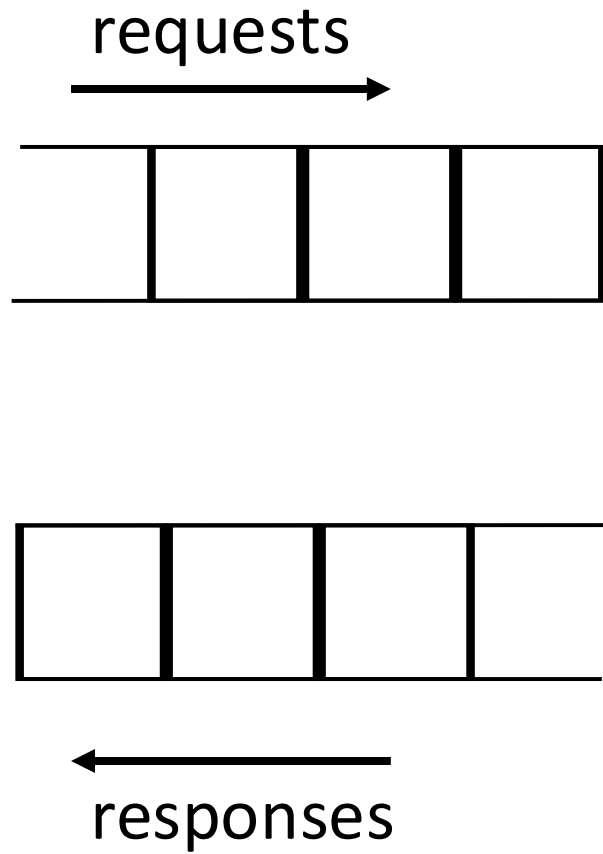
3. Hardware acceleration



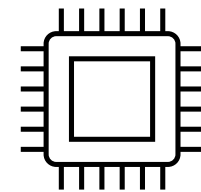
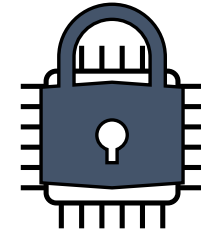
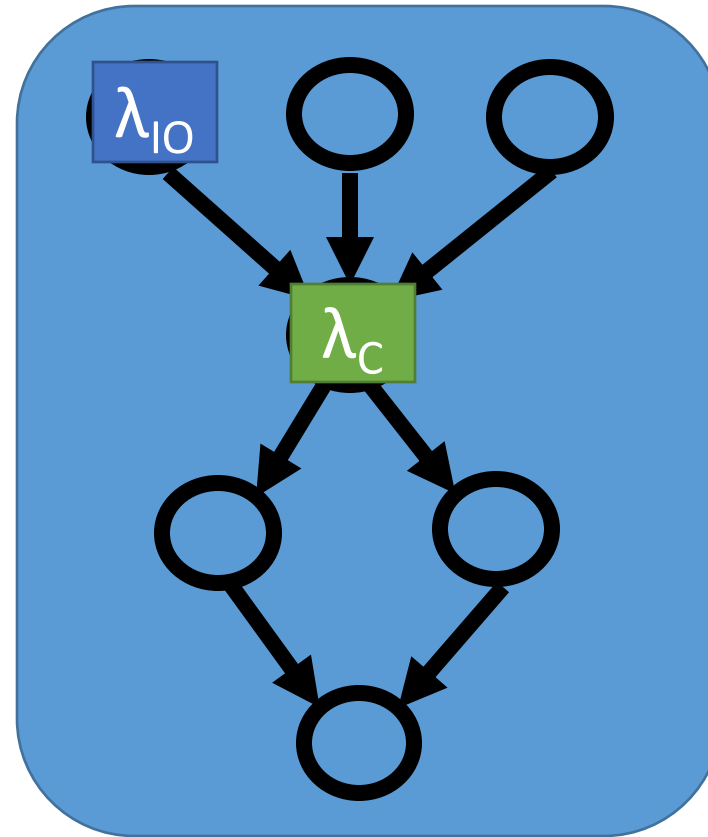
Dandelion Dispatcher



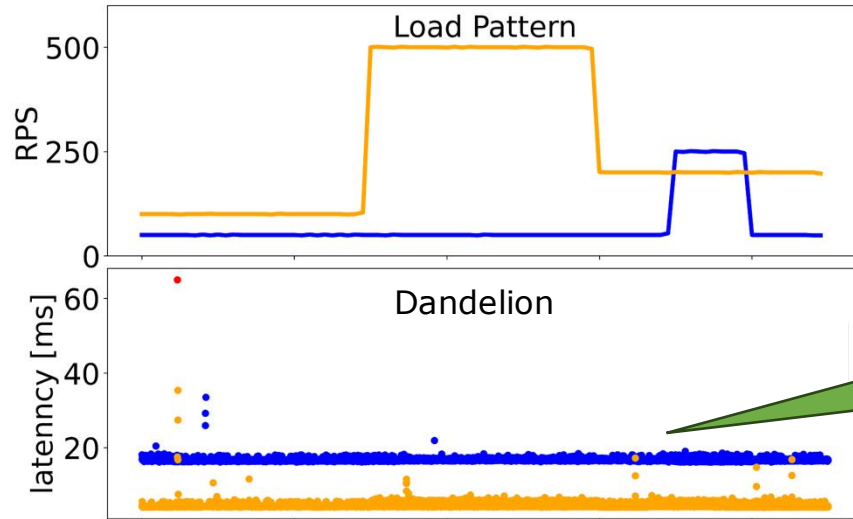
3. Hardware acceleration



Dandelion Dispatcher



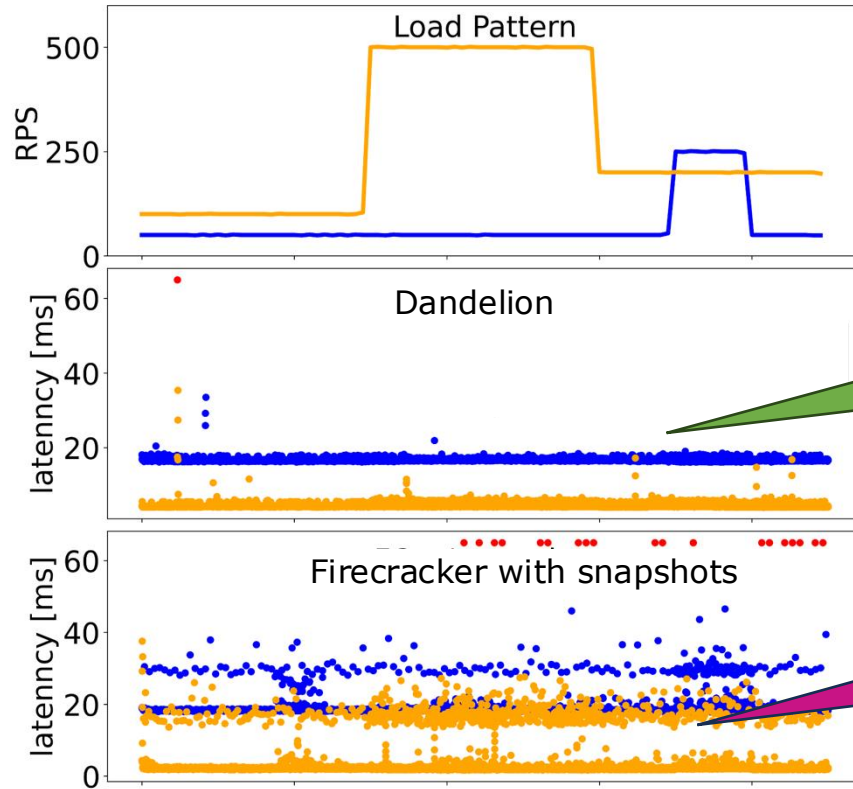
Application performance



- img compression → compute-intensive
- log processing → I/O-intensive

Dandelion maintains stable, low latency, adjusting the # compute vs. comm engines as the app mix changes

Application performance

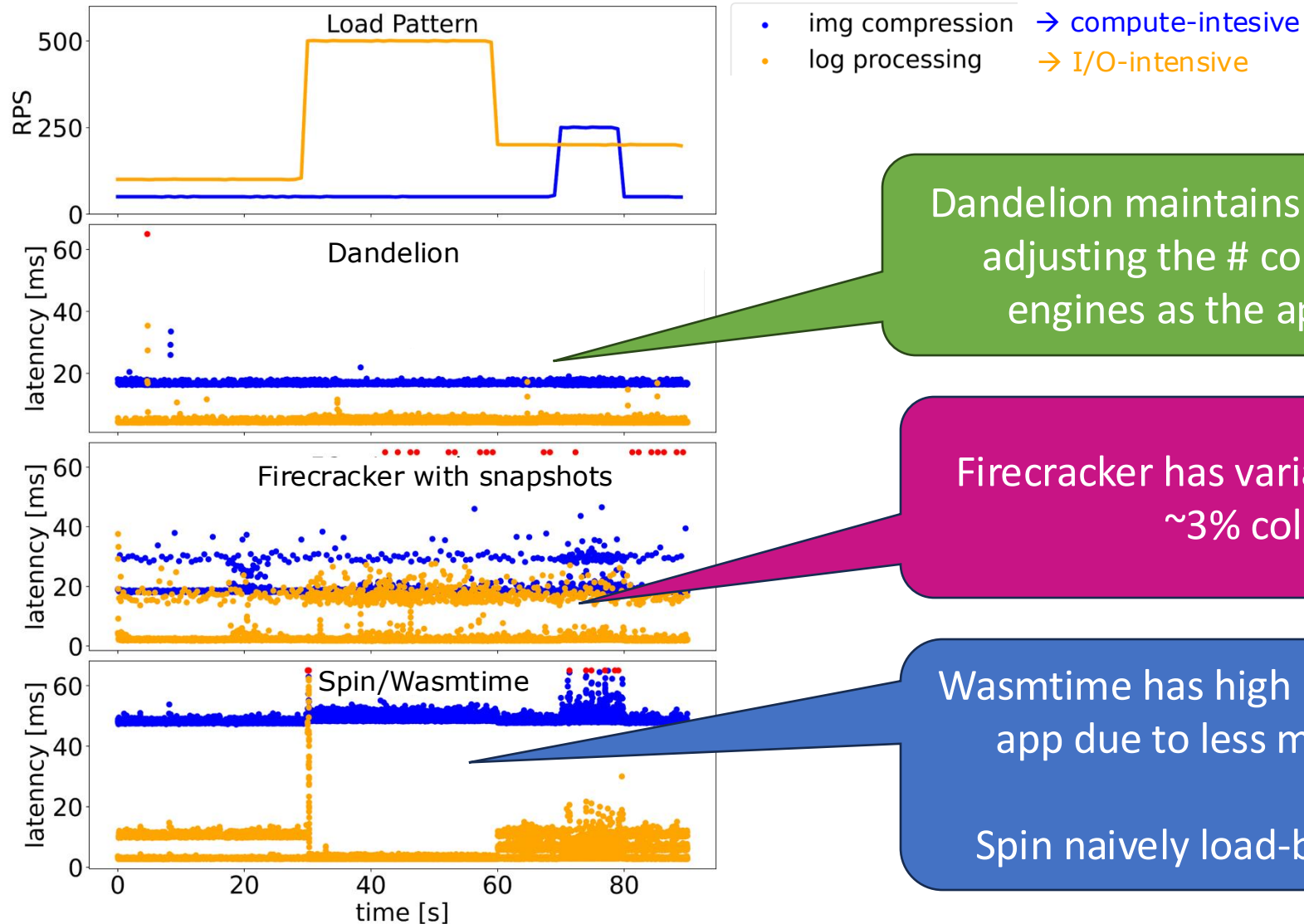


- img compression → compute-intensive
- log processing → I/O-intensive

Dandelion maintains stable, low latency, adjusting the # compute vs. comm engines as the app mix changes

Firecracker has variable latency due to ~3% cold starts

Application performance



Dandelion maintains stable, low latency, adjusting the # compute vs. comm engines as the app mix changes

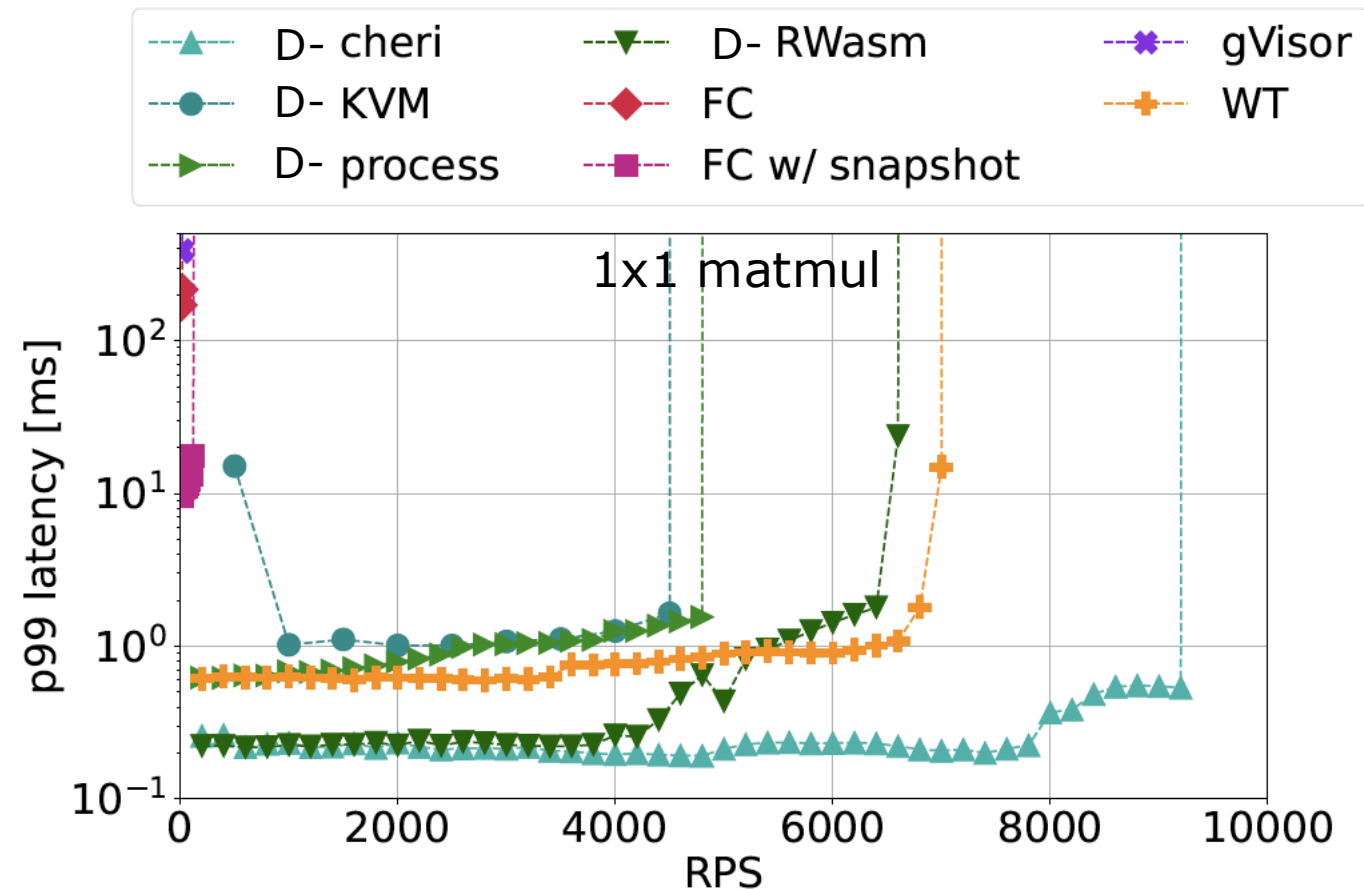
Firecracker has variable latency due to ~3% cold starts

Wasmtime has high latency for compute app due to less mature compilers.

Spin naively load-balances requests.

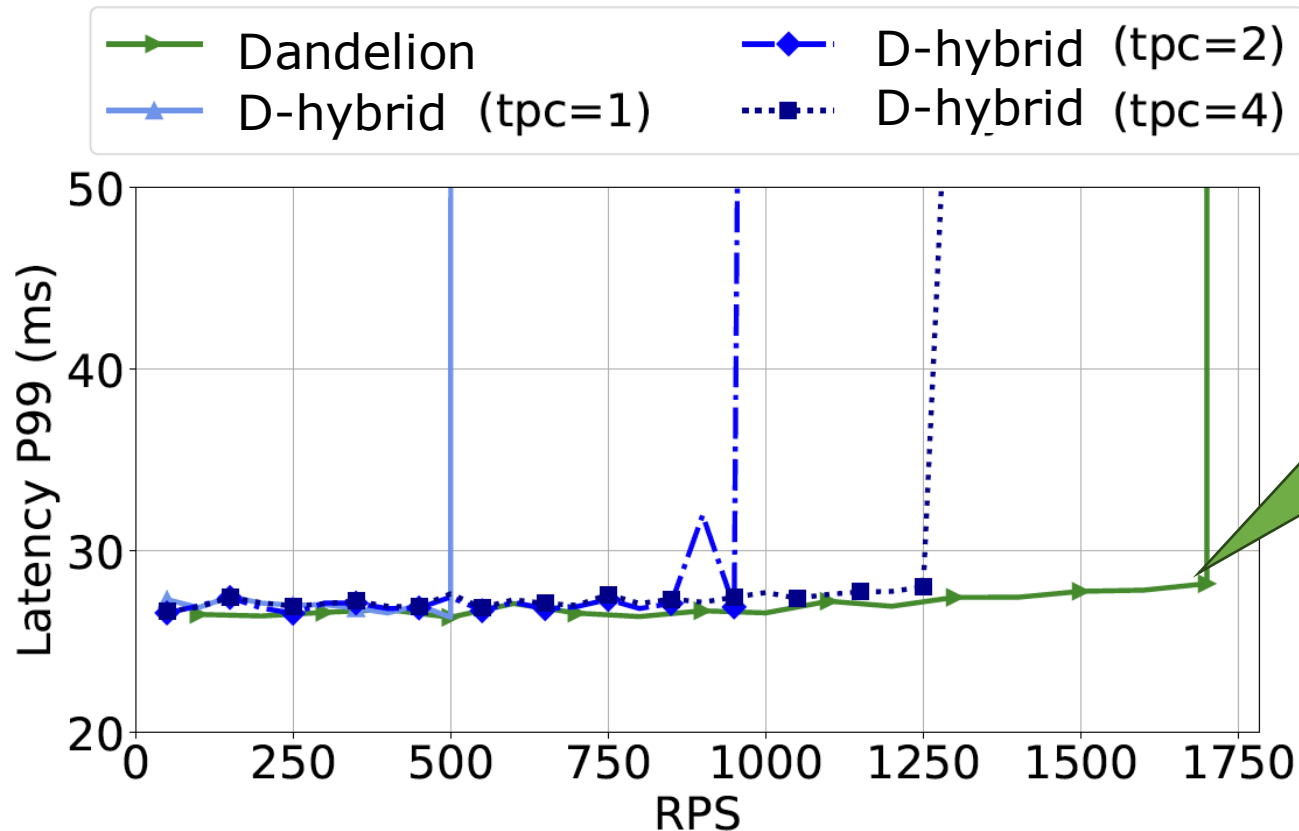
Fast cold starts

Dandelion creates sandbox in $90\mu\text{s}$ to $640\mu\text{s}$ $\rightarrow$ practical to always cold start



CPU scheduling with pure vs. hybrid functions

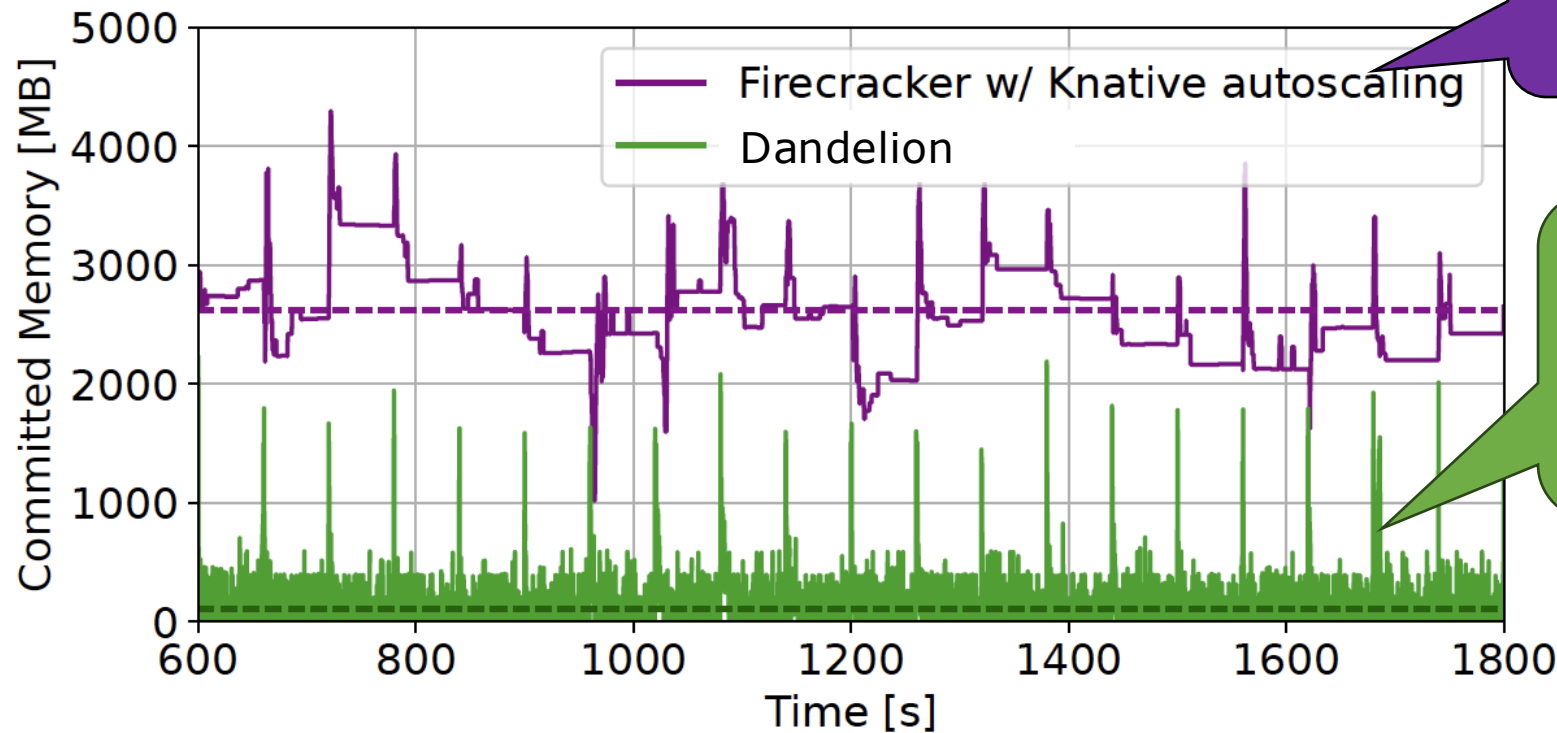
- Dandelion achieves higher throughput by splitting compute & comm vs. a modified version that executes “hybrid” compute+comm functions



Separating compute vs. communication functions improves CPU scheduling
→ higher peak throughput

Dandelion only uses memory the app needs

Azure Functions trace replay with 100 functions



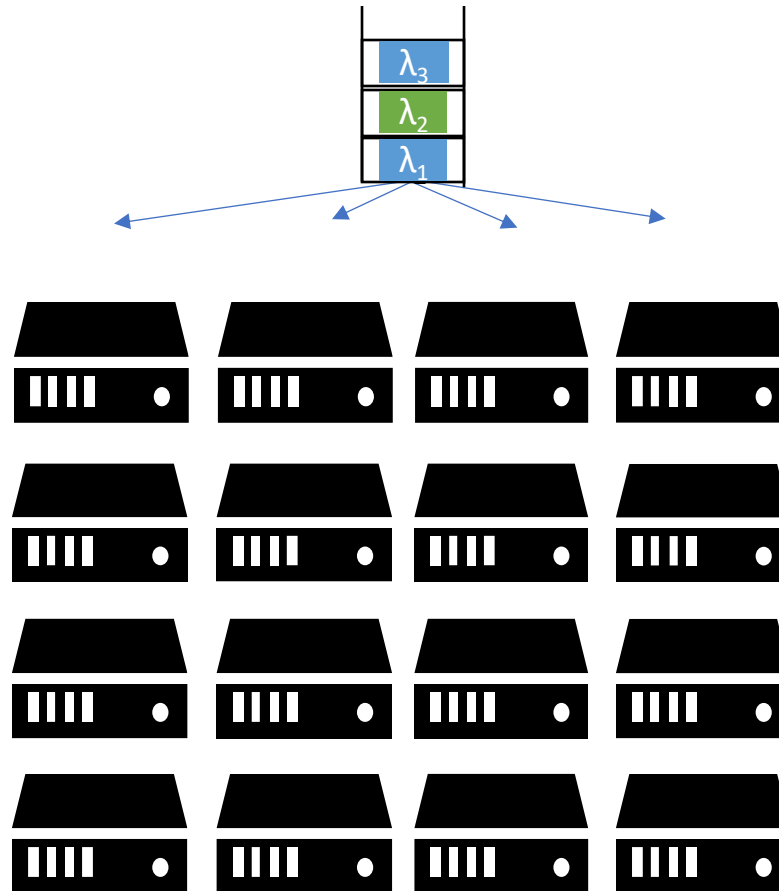
Assume default Knative autoscaling policy for Firecracker server

Dandelion saves memory by not keeping hot VMs
→ 96% memory savings

Fast cold start on a server is not enough...



Also need fast scheduling of requests to servers...



Current approach: retrofit legacy cluster manager

Adds “serverless” features:

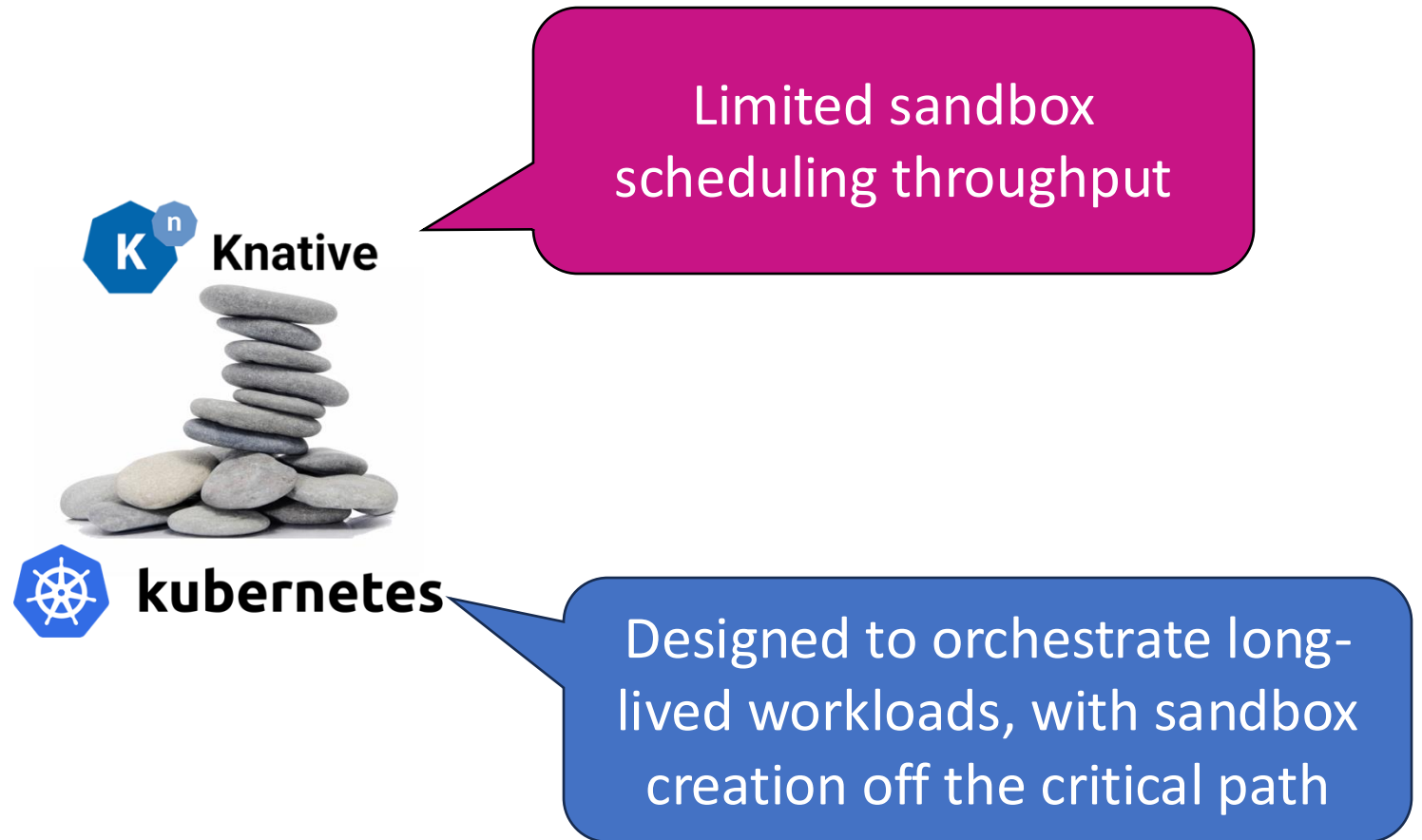
- Autoscaling sandboxes based on request load
- Scale-to-zero support → buffering requests while wait for sandbox creation



Provides useful functionality:

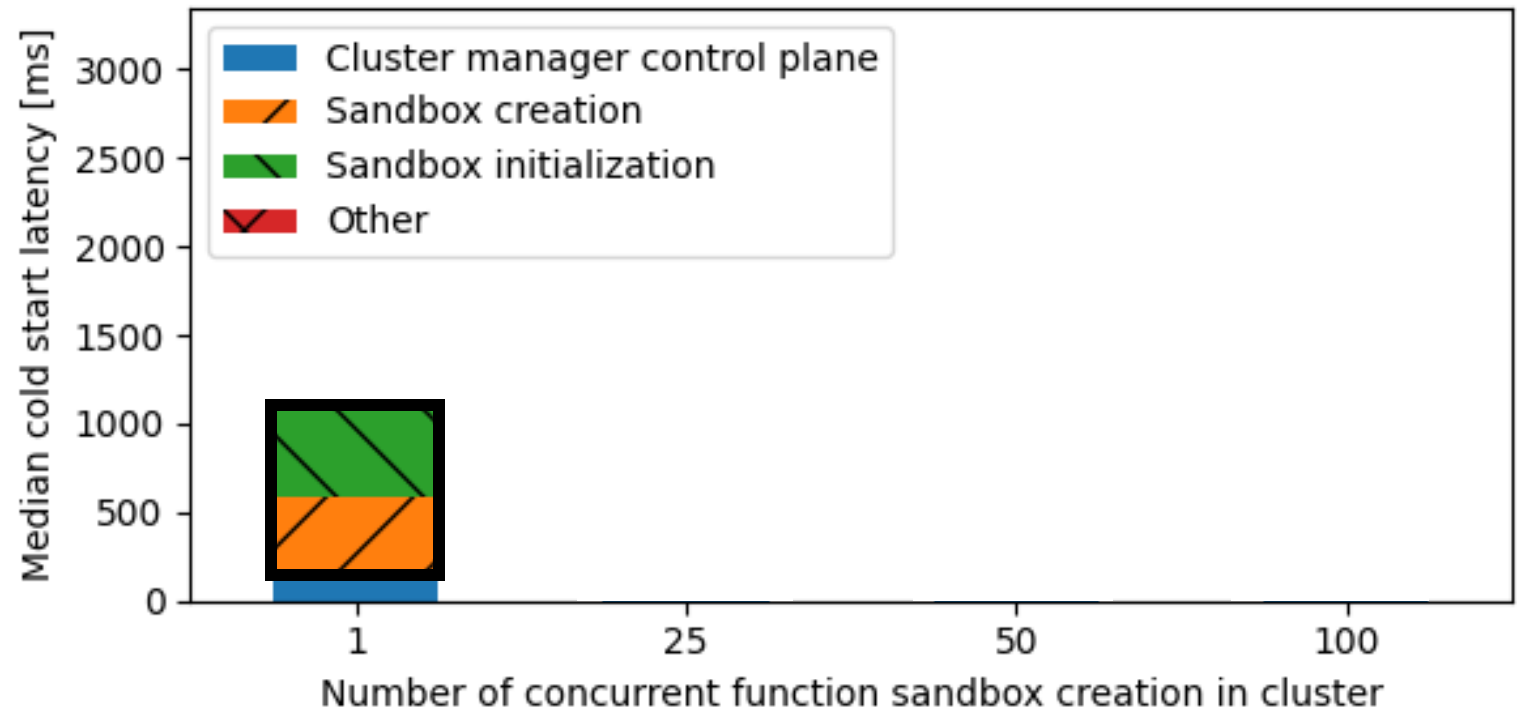
- Request load balancing
- Server and sandbox health monitoring
- Autoscaling sandboxes based on CPU/mem utilization
- Fault tolerance

Current approach: retrofit legacy cluster manager



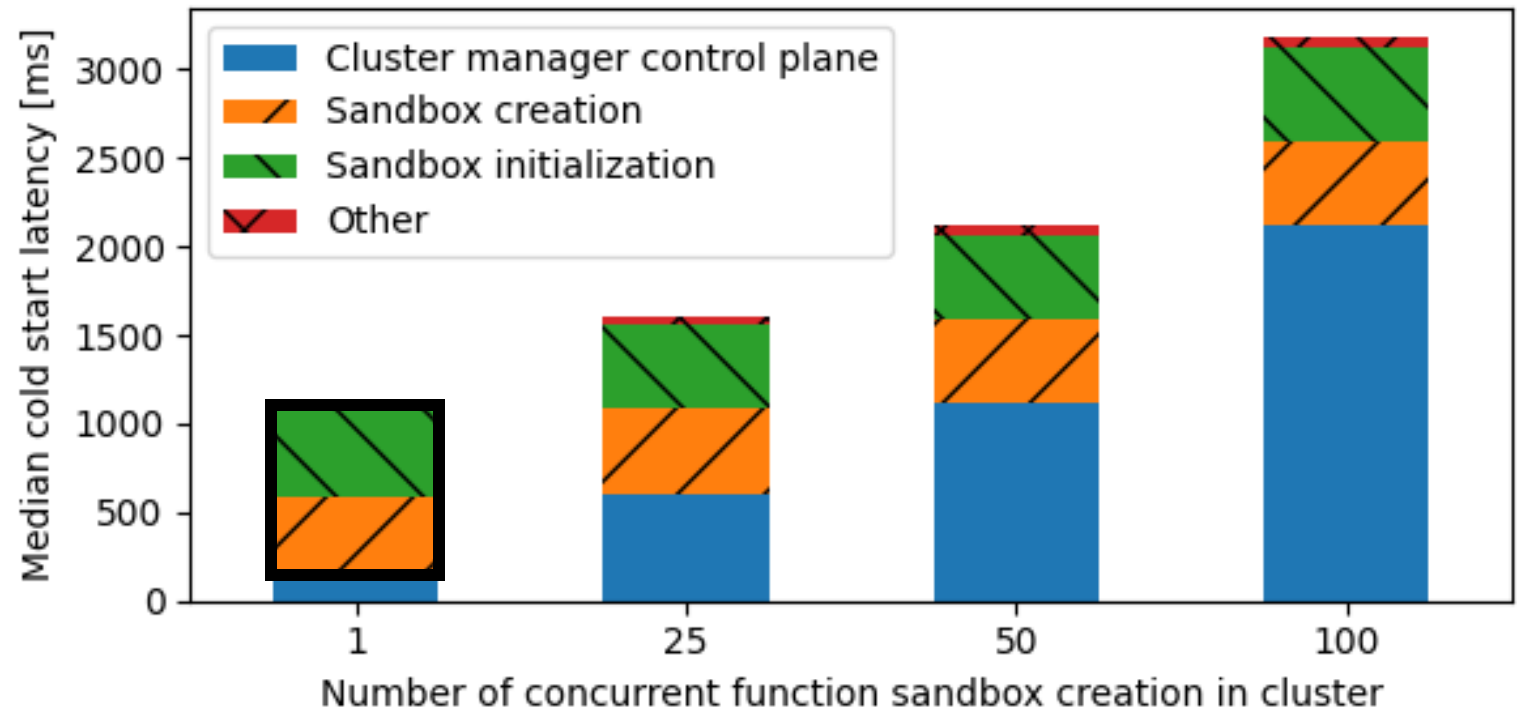
Cold start end-to-end latency breakdown

- Knative-on-K8s



Cold start end-to-end latency breakdown

- Knative-on-K8s



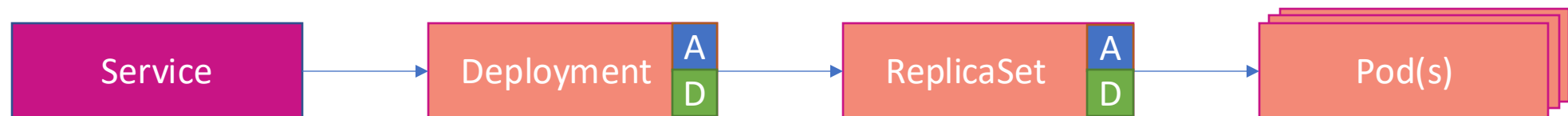
The cluster manager scheduling overhead dominates at scale.

Where does cluster manager overhead come from?

1. Complex hierarchical cluster state

For example, Knative:

- Associates each function with multiple hierarchical K8s abstractions
- Tracks and reconciles **actual** and **desired state** for each abstraction with a controller

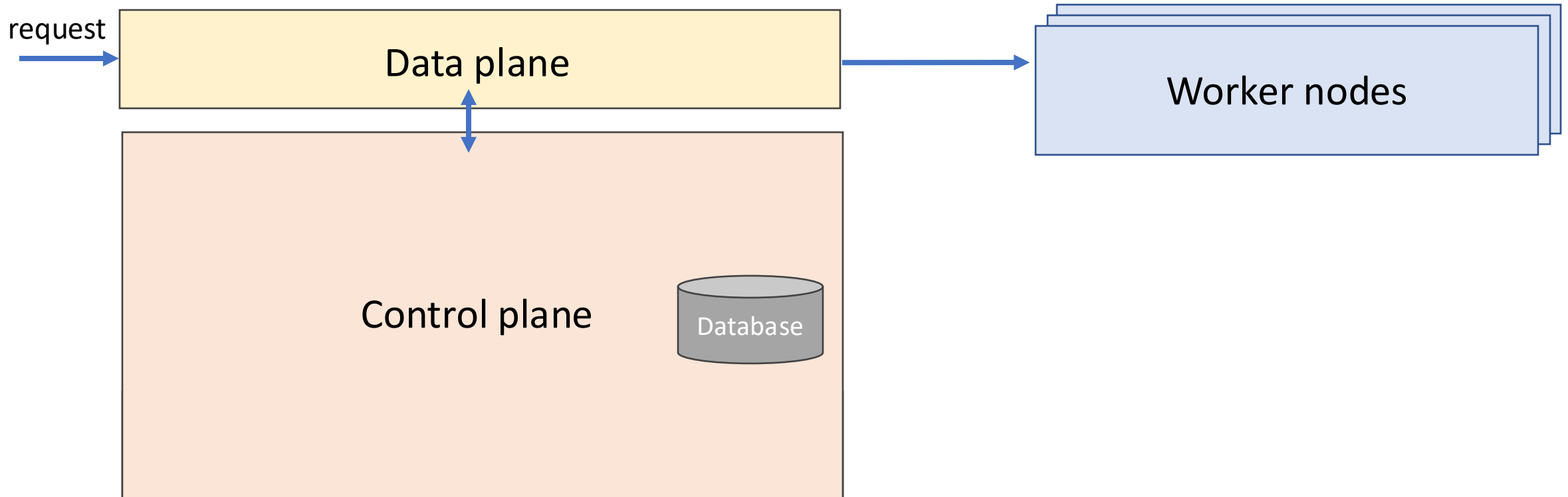


Where does cluster manager overhead come from?

1. Complex hierarchical cluster state

2. Persist all cluster state updates

3. Microservice architecture with per-sandbox sidecars

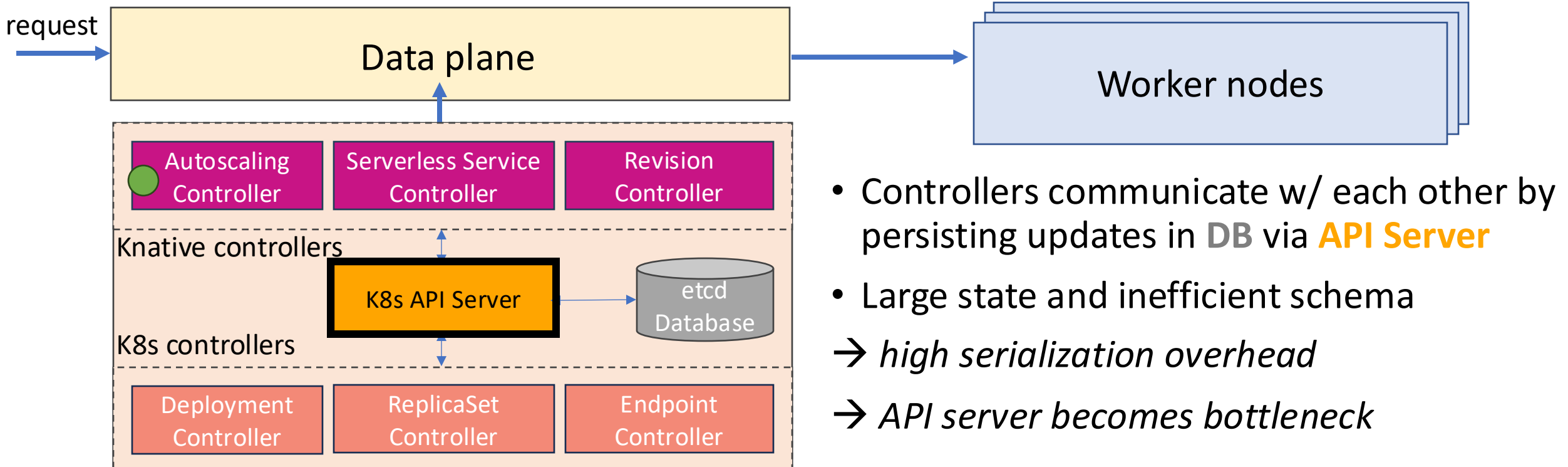


Where does cluster manager overhead come from?

1. Complex hierarchical cluster state

2. Persist all cluster state updates

3. Microservice architecture with per-sandbox sidecars

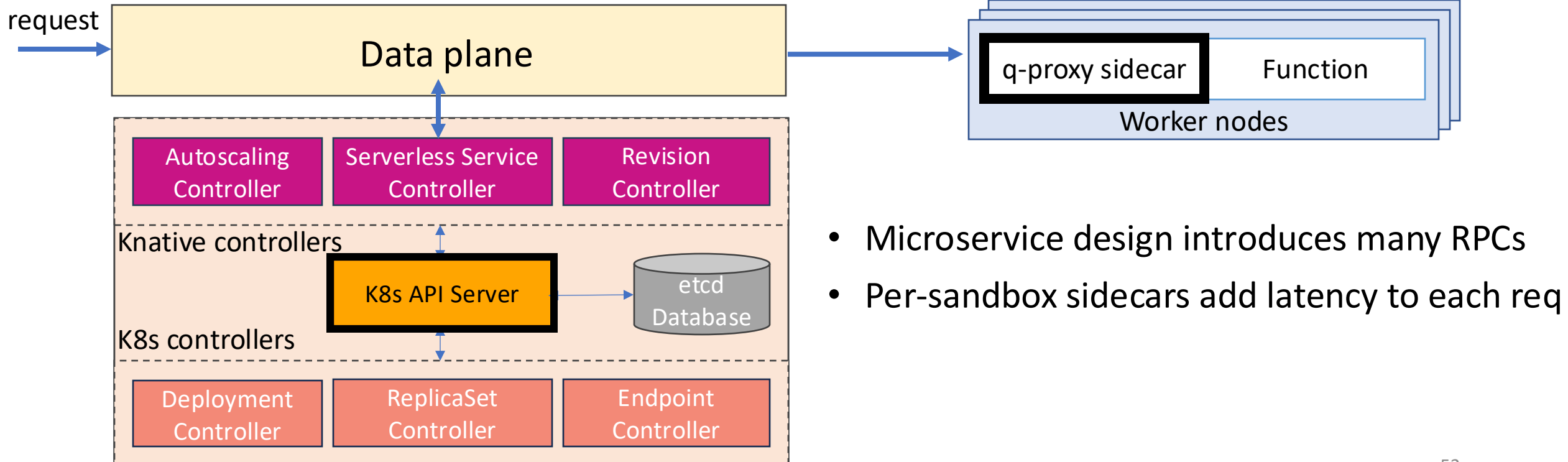


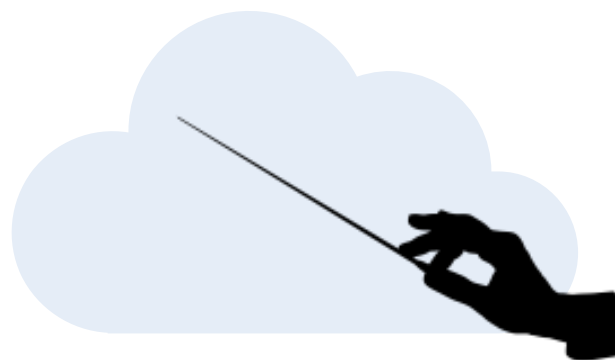
Where does cluster manager overhead come from?

1. Complex hierarchical cluster state

2. Persist all cluster state updates

3. Microservice architecture with per-sandbox sidecars





Dirigent



Lazar Cvetković



François Costa



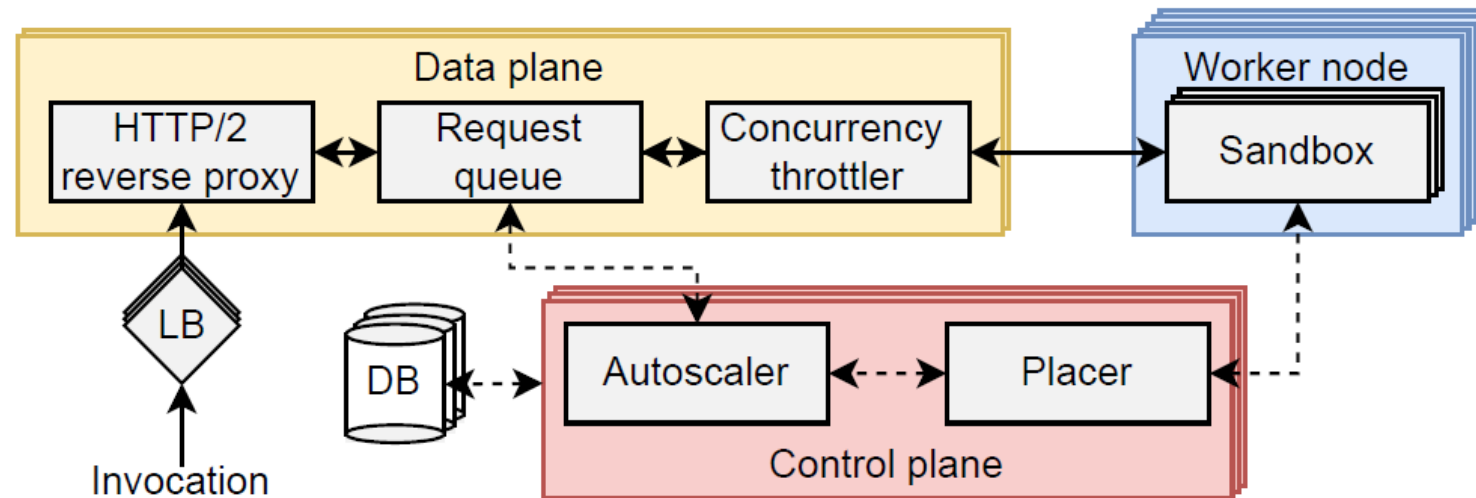
Mihajlo Djokic



Michal Friedman

Dirigent

- A new clean-slate cluster manager for serverless computing
- Standalone research system with simple architecture



Challenges and solutions

1. Complex hierarchical cluster state

2. Persist all cluster state updates

3. Microservice architecture with per-sandbox sidecars



Simple internal
abstractions

Simple internal abstractions

- Eliminate hierarchy of abstractions
 - E.g., deployments, replica sets, pods
- Simplify cluster state schema

Abstraction	Associated State
Function	Name
	Image URL
	Port to expose
	Scheduling config
	Scheduling metrics
Sandbox	Name
	IP address
	Port on worker node
	Worker node ID
DataPlane	IP address
	Port
WorkerNode	Name
	IP address
	Port

Challenges and solutions

1. Complex hierarchical cluster state



Simple internal abstractions

2. Persist all cluster state updates



Persistence-free latency-critical operations

3. Microservice architecture with per-sandbox sidecars

Persistence-free latency-critical operations

Abstraction	Associated State
Function	Name
	Image URL
	Port to expose
	Scheduling config
	Scheduling metrics
Sandbox	Name
	IP address
	Port on worker node
	Worker node ID
DataPlane	IP address
	Port
WorkerNode	Name
	IP address
	Port

Persistence-free latency-critical operations

- Dirigent persists only bare-minimum state required for fault tolerance
 - The rest can be reconstructed
 - Relax exact cluster state reconstruction:
 - Sandboxes may get created on different node, different IP address
- OK for serverless (hidden from users)

Abstraction	Associated State	Persisted
Function	Name	✓
	Image URL	✓
	Port to expose	✓
	Scheduling config	✓
	Scheduling metrics	
Sandbox	Name	
	IP address	
	Port on worker node	
	Worker node ID	
DataPlane	IP address	✓
	Port	✓
WorkerNode	Name	✓
	IP address	✓
	Port	✓

Challenges and solutions

1. Complex hierarchical cluster state



Simple internal
abstractions

2. Persist all cluster state updates



Persistence-free latency-
critical operations

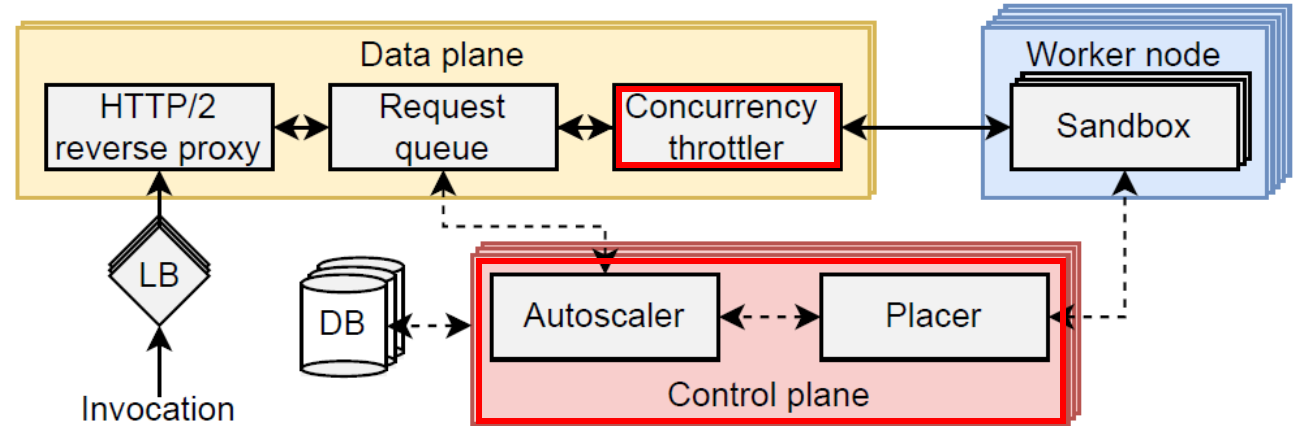
3. Microservice architecture with per-sandbox sidecars



Monolithic
control and data plane

Monolithic control and data planes

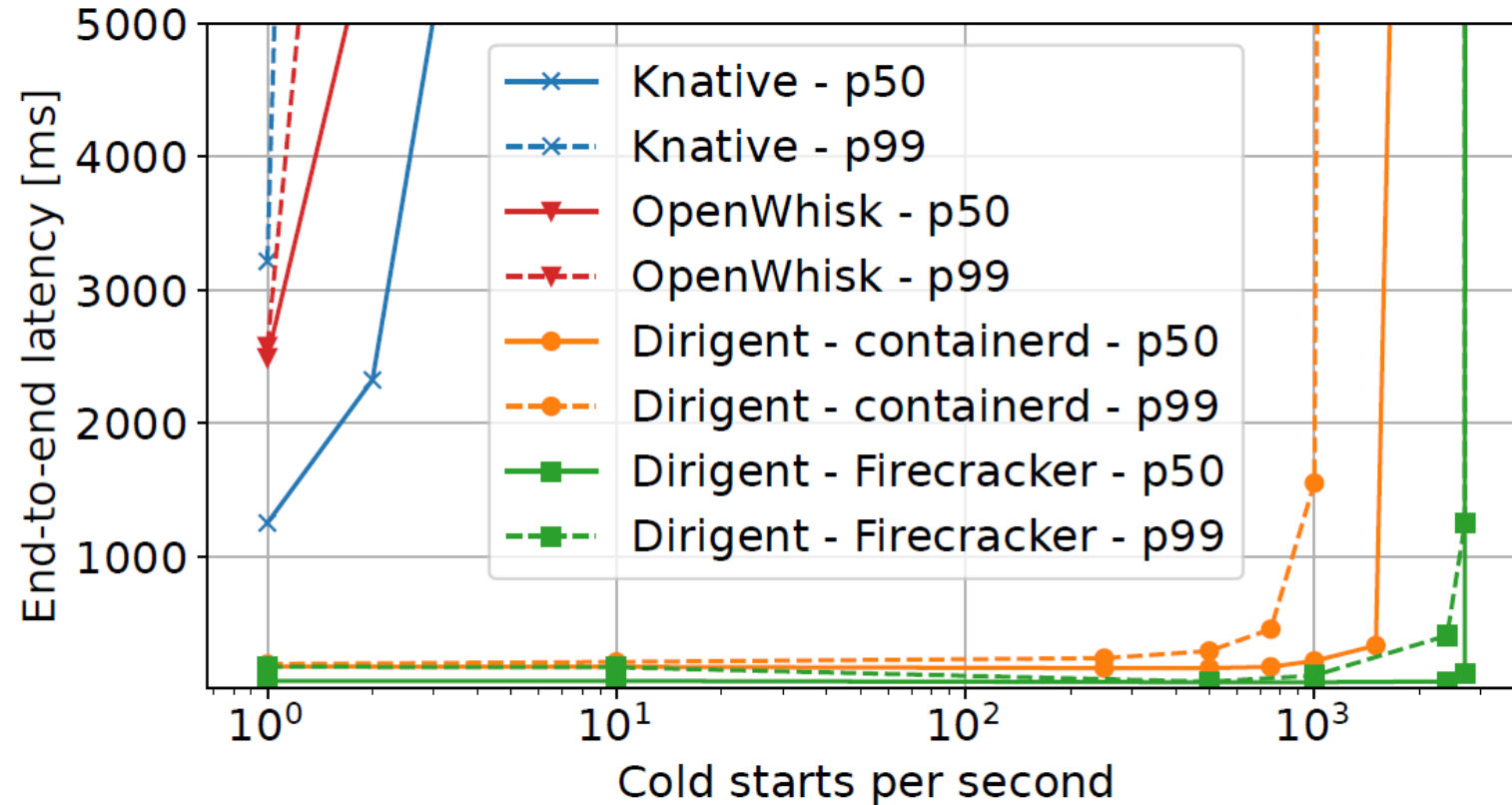
- Centralized control plane
 - Easy management and simple deployment
 - Dirigent can scale further by federating clusters
- Centralized concurrency throttling
 - No per-sandbox sidecars
 - Less network stack traversals



Evaluation

- Experiment setup
 - 100-node cluster (93 nodes are workers)
 - 10-physical core xl170 Cloudlab machine with 64 GB DRAM and SSD
- Baselines:
 - Knative v1.13.1 on K8s v1.29.1
 - OpenWhisk v1.0.1 on K8s v1.29.1
- Sandbox runtimes: containerd and Firecracker (w/wo snapshots)
- Workload: hello-world and busy looping functions
- Highly-available cluster

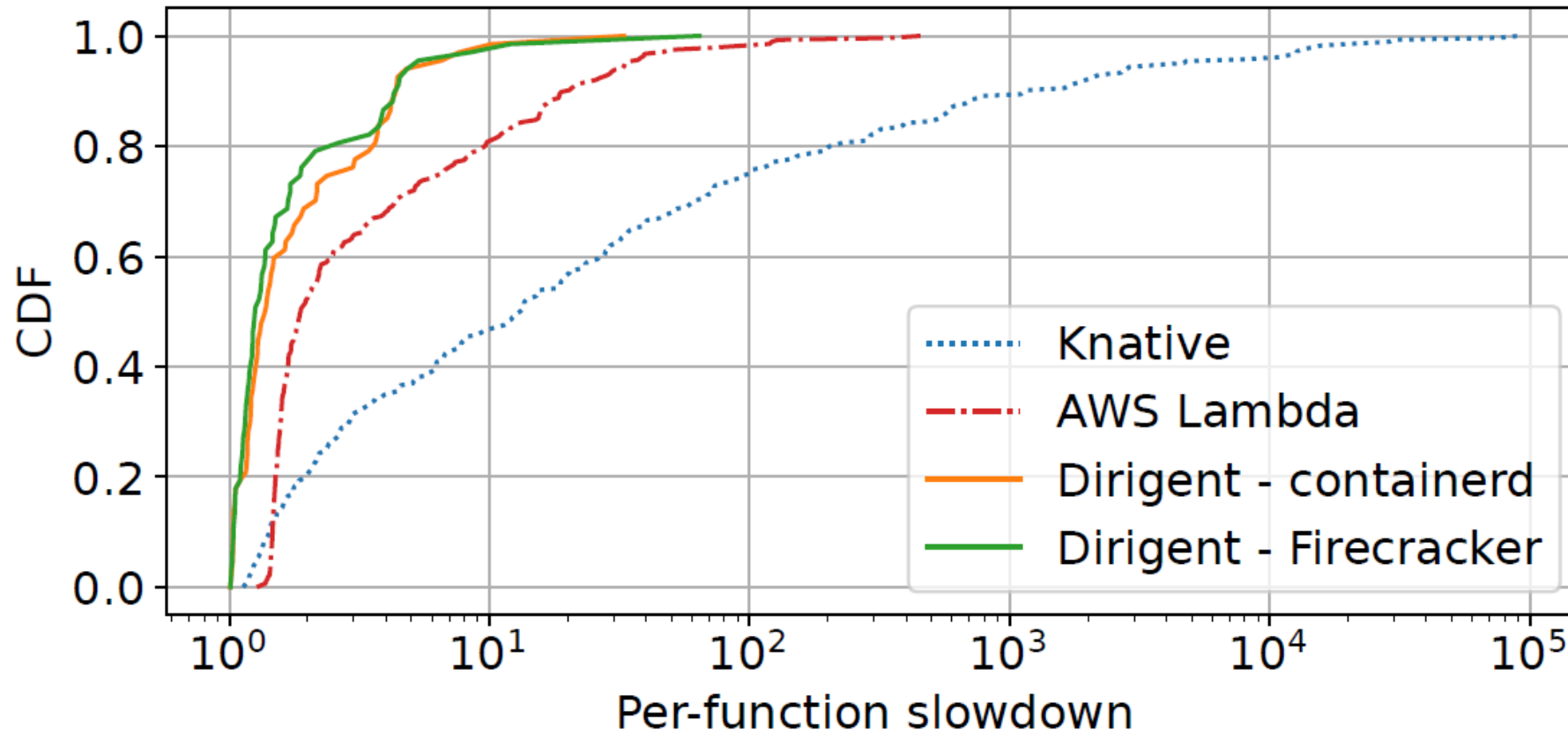
Cold start performance



Dirigent achieves 1250x higher cold start throughput compared to Knative

Real workload performance: Azure trace sample

- Metric: $\text{slowdown}_f = \text{avg}(\text{e2e_latency}_f / \text{exec_time}_f) = \text{scheduling overhead}$



Dirigent achieves lower per-function slowdown

How practical is all of this?

How practical is it to replace cluster manager?

- The implementation of the cluster manager is hidden from users, so it requires no changes to user applications
- Cloud providers have incentive to optimize resource efficiency → cost

How practical is it to introduce a new programming model for the cloud?

- AWS Lambda introduced new Functions as a Service programming model in 2014, now all major providers have a similar offering

How practical is it to introduce a new programming model for the cloud?

- AWS Lambda introduced new Functions as a Service programming model in 2014, now all major providers have a similar offering
- Recent works propose programming models similar to Dandelion

Computation-Centric Networking

Yuhan Deng, Angela Montemayor, Amit Levy*, and Keith Winstein
Stanford University, *Princeton University

Abstract
We propose putting *computation* at the center of cloud service architectures. We envision a shared representation of a computation, run in an environment of shared resources. This suggests an end-to-end programming model, shifting the service model from “providing the hardware” to “providing the untrusted execution of a computation.” Accountability and security considerations could permit agility and innovation.

Figure 1. Example of specifying data intent for a cloud function that generates the thumbnail of an image. It reads input from pic/1.jpg and writes output to thumb/1.jpg.

```
thumbnail(params={
  "get_data": ["pic/1.jpg"],
  "put_data": ["thumb/1.jpg"]
})
```

Figure 1. Example of specifying data intent for a cloud function that generates the thumbnail of an image. It reads input from pic/1.jpg and writes output to thumb/1.jpg.

DataFlower: Exploiting the Data-flow Paradigm for Serverless Workflow Orchestration

Zijun Li*
Shanghai Jiao Tong University
Shanghai, China
ljzx1122@sjtu.edu.cn

Jieru Zhao
Shanghai Jiao Tong University
Shanghai, China
zhao-jieru@sjtu.edu.cn

ABSTRACT
Serverless computing that runs functions with automatic task execution pattern in the cloud-native environment. It integrates serverless functions into workflows, tenants can orchestrate a serverless workflow. However, existing paradigm inherently results in long response latency due to heavy data persistence overhead, sequential resource allocation, and late function triggering.

Lambdata: Optimizing Serverless Computing by Making Data Intents Explicit

Yang Tang
Department of Computer Science
Columbia University
New York, NY, USA
Email: ty@cs.columbia.edu

Junfeng Yang
Department of Computer Science
Columbia University
New York, NY, USA
Email: junfeng@cs.columbia.edu

Abstract—Serverless computing emerges as a new paradigm to build cloud applications, in which developers write small functions that react to cloud infrastructure events, and cloud providers maintain all resources and schedule the functions in containers. Serverless computing thus enables developers to focus on their core business logic and leave server management and scaling to cloud providers. Unfortunately, existing serverless computing systems suffer from a key limitation that deprives them of enjoying significant

How practical is it to introduce a new programming model for the cloud?

- AWS Lambda introduced new Functions as a Service programming model in 2014, now all major providers have a similar offering
- Recent works propose programming models similar to Dandelion
- Need to make life easy for programmers → automate porting
 - Can leverage compilers to extract compute & I/O segments of applications

How practical is it to introduce a new programming model for the cloud?

- AWS Lambda introduced new Functions as a Service programming model in 2014, now all major providers have a similar offering
- Recent works propose programming models similar to Dandelion
- Need to make life easy for programmers → automate porting
 - Can leverage compilers to extract compute & I/O segments of applications
- The performance and efficiency benefits need to be compelling

(How) does it help for AI applications?

- So far, Dandelion and Dirigent designed for “general” cloud applications
- AI is an important, power-hungry class of cloud applications
 - Worth optimizing for explicitly, with custom systems and programming models
- Opportunity to think more holistically about a cloud programming model for AI applications and general applications
 - Dandelion as a declarative programming model for execution on heterogeneous HW
 - Agentic AI workflows → Dandelion compositions



Conclusion

- The cloud needs to provide a “**serverless**” abstraction to users such that the platform can optimize resource allocations for energy efficiency
- The current approach of retrofitting existing cloud abstractions and systems (e.g., VMs, cluster managers) for serverless is not efficient
- We need to re-think the computer system stack with energy-efficiency in mind
 - **Dandelion** programming model exposes dataflow to the platform (enabling efficient scheduling) and reduces user code attack surface (allowing execution in lean sandboxes)
 - **Dirigent** re-architects cluster management for high scheduling throughput